產品功能開發說明書

AI Dev Skills 技能包・多 AI 安裝程式・AI Dev 進度看板

平台定位：桌機＝完整操作平台｜手機＝監看（唯讀）

項目	內容
文件編號	AIAD-PRD-BOARD-001
產品負責人	Kuan Yu
讀者	產品負責人、開發 AI 代理（高階／中低階）、驗證與審查代理、工程師
文件狀態	草稿（v0.1）— 待產品負責人審閱核准
依據來源	aiad-skills-installer 原始碼（board_src、hooks_src、enforce_src、install.ps1、build_payload.py、gen_nsi.py）、dist/guide.html、ai-dev-* 五個 SKILL.md
標示規則	**【現況】**＝程式已實作；**【建議】**＝本文件新增提案；**【待決】**＝需產品負責人決定；**【未驗證】**＝尚未在目標環境實測

版本紀錄

版本	日期	作者	說明
v0.1	2026-11-01	Grok Bot（為 Kuan Yu 撰寫）	初稿：依現有程式與指南整理功能規格；新增手機監看版設計（建議）與待決事項
v0.2	（待定）	—	產品負責人審閱後修訂；決定第 14 章待決事項

目錄

1 產品概述 4
1.1 產品目標 4
1.2 使用者角色 4
1.3 平台定位：桌機完整操作，手機監看 4
2 範圍與非範圍 6
2.1 範圍（In Scope） 6
2.2 非範圍（Out of Scope） 6
3 系統架構 7
3.1 整體架構 7
3.2 元件說明 7
3.3 安裝根目錄檔案配置 8
3.4 主要資料流 8
4 功能規格 9
4.1 AI 開發流程技能包 9
4.2 多 AI 工具安裝程式 10
4.3 進度看板：工作層級與欄位 12
4.4 健康燈號與燈號篩選 14
4.5 即時更新（JSON 監看 → SSE） 16
4.6 Inspector、時間戳、歷程與備註 17
4.7 佈署與版本卡（Deployment） 19
4.8 帳號、登入與授權 21
4.9 首次啟動與新增專案 23
4.10 多專案切換 25
4.11 board.json 版本歷史（Git 自動 commit） 26
4.12 PR／CI 通知 27
4.13 Git 分頁：瀏覽 28
4.14 Git 分頁：操作（僅桌機） 30
4.15 強制遵守機制（四層） 34
5 資料模型 36
5.1 board.json（version 1） 36
5.2 設定檔（.ai-board，不進 Git） 36
5.3 其他設定 37
6 API 規格 38
6.1 公開與驗證 38
6.2 專案 API（/api/p/{id}/...） 38
6.3 錯誤碼（節錄） 39
7 權限矩陣 40
8 安全性 41
8.1 現況控制 41
8.2 手機／遠端存取的安全要求【建議】 41
9 非功能需求 42
10 手機監看版設計【建議】 43
10.1 設計原則 43
10.2 斷點與版面 43

10.3 手機功能清單 44
10.4 通知 44
10.5 手機登入 45
10.6 遠端存取方案（LAN／Tunnel） 45
10.7 手機監看 BDD 45
11 測試策略 47
11.1 原則 47
11.2 測試層級 47
11.3 CI 關卡 47
12 部署與安裝 48
12.1 建置 48
12.2 安裝與首次使用流程 48
12.3 升級與移除 48
12.4 手機監看上線（建議） 48
13 已知限制與未驗證項目 49
14 待決事項 50
附錄 A 名詞解釋 51

1 產品概述

1.1 產品目標

本產品是一套「用 AI 代理把產品從規劃做到 PR」的開發流程工具組，由三個部分組成：

組成	說明
AI 開發流程技能包	五個 Skill（ai-dev-pipeline、preproduction、implementation、pr-acceptance、production-readiness），把開發拆成預製期 → 實作期 → PR 驗收期，每個關卡都要有證據才放行。
多 AI 安裝程式	把技能轉成 Cursor、Codex（AGENTS.md）、Claude Code、GitHub Copilot、Windsurf、Gemini CLI、Cline／Roo Code 各自讀得懂的格式，並安裝 AI Hooks、Git Hooks、CI Guard 與看板執行層。提供 NSIS 安裝檔（AI-Dev-Skills-Setup.exe）與 PowerShell zip 兩種形式。
AI Dev 進度看板	零相依的本機看板伺服器（serve-board.ps1）與單頁 HTML（board.html）：即時顯示 docs/board.json、五色健康燈號、Inspector、版本歷史、多專案、帳號與權限、PR/CI 通知，以及 Git 分頁（瀏覽與操作）。

產品目標：

- 讓產品負責人在一個畫面掌握「規劃 → 實作 → 驗收 → 佈署」全程的進度與風險（健康燈號）。
- 讓不同等級的 AI 模型各司其職，且**開發者與驗證者必須是不同代理**；只有高階模型或使用者能標完成。
- 用四層強制機制（規則 → AI Hooks → Git Hooks／CI Guard → 分支保護）讓 AI 無法繞過流程。
- 零相依、可在 Windows PowerShell 5.1 單機執行，資料以 JSON 檔與 Git 保存，可追溯每一版。

1.2 使用者角色

角色	代表	主要目標	看板角色值
產品負責人／使用者	Kuan Yu	核准規格與 BDD、監看進度與風險、核准發版、管理帳號與專案	admin
高階 AI 模型（架構師／審查者）	高階模型代理	預製期規劃、根因診斷、獨立審查並提交 PR、標記完成	high-tier
開發代理（中低階）	一張工單一個代理	依 BDD+TDD 實作工單，只能在 feature/* 分支操作	dev-agent
驗證代理（中低階，另開）	獨立啟動的代理	依 TDD 實際操作介面、截圖錄影，記錄 PASS/FAIL/NOT RUN	verifier
檢視者	利害關係人、PM	只看進度，不能操作	viewer

1.3 平台定位：桌機完整操作，手機監看

產品負責人要求（本文件的平台基準）

桌機（Desktop）是完整操作平台：所有功能（看板、Inspector、歷史、帳號管理、專案新增、Git 瀏覽與所有 Git 寫入操作、合併編輯器、互動式 rebase）只在桌機提供。

手機（Mobile）只做監看：唯讀檢視、通知、健康燈號、PR/CI 狀態。**手機上不提供任何編輯，也不提供任何 Git 寫入操作**。

【現況】伺服器只綁定 `http://localhost:8765/`，手機目前無法連線；手機連線方式為**【建議】**並列為**【待決】**（見第 10 章、第 14 章）。

面向	桌機	手機
定位	完整操作	監看（唯讀）
看板與 Inspector	完整五欄、Inspector 側欄、篩選、歷史版本	燈號摘要、需注意清單、卡片唯讀詳細（單欄）
通知	右上角浮動通知、🔔 最近 100 筆、瀏覽器桌面通知	頁面內 🔔 清單；**【建議】**Web Push 推播
PR／CI	Git 分頁完整 PR 清單與檢查項目	PR 狀態、CI 結果、審查結果（唯讀）
Git	線圖、詳細、分支、所有寫入操作	不提供 Git 分頁寫入；**【建議】**僅顯示分支與 PR 摘要
帳號／專案管理	管理者可核准、改角色、新增專案、設定 Client ID	不提供
連線	localhost（現況）	**【建議】**選用 LAN（HTTPS＋Token）或 Tunnel

2 範圍與非範圍

2.1 範圍（In Scope）

- 五個 AI 開發流程技能與其在七種 AI 工具的安裝格式。
- NSIS 安裝檔與 PowerShell zip 安裝／移除、備份 .bak、清單 manifest、從舊版升級的設定遷移。
- 進度看板：工作層級、欄位、健康燈號、燈號篩選、即時更新、Inspector、時間戳、歷程、備註、佈署卡。
- 看板伺服器：GitHub Device Flow 登入、本機備用帳號、角色與專案授權、待核准流程、多專案、board.json 自動 commit 與歷史、遠端同步、PR/CI 通知。
- Git 分頁：概覽、線圖、commit 詳細、分支、PR、搜尋；Git 寫入操作與安全規則、操作紀錄。
- 強制遵守：AI Hooks（Cursor、Claude Code）、Git Hooks、GitHub Actions CI Guard、分支保護說明。
- 首次啟動流程（先啟動再於網頁設定管理者／Client ID，再新增專案）。
- 【建議】手機監看版（響應式版面、唯讀、通知、遠端存取方案）。

2.2 非範圍（Out of Scope）

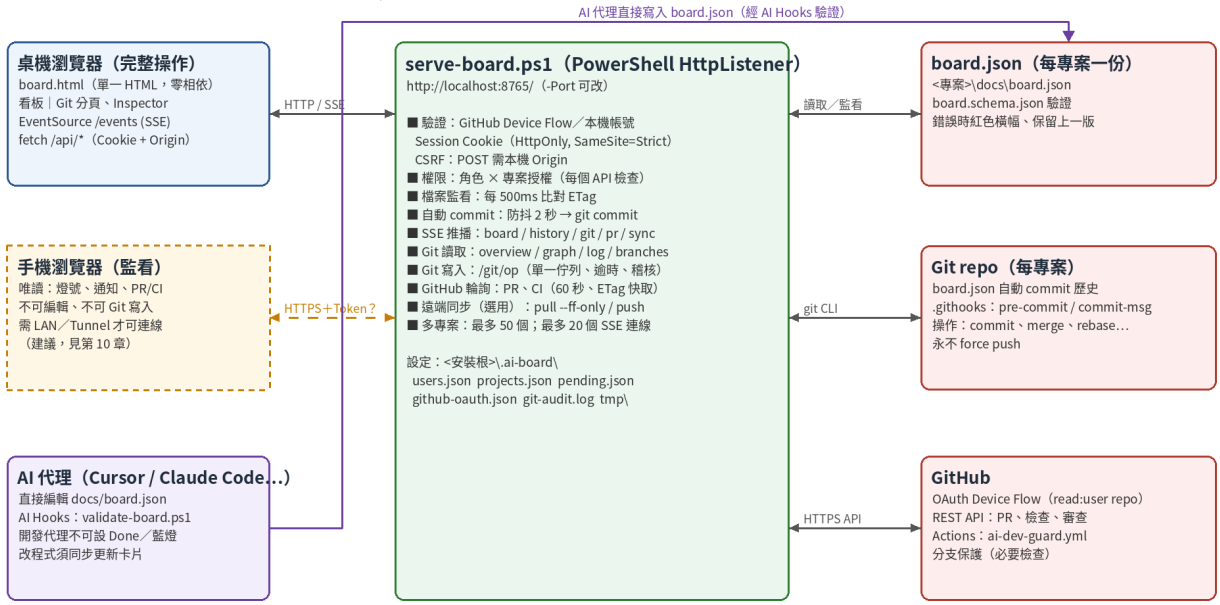
- **看板上的線上編輯卡片**：看板沒有線上編輯，代理照舊修改 board.json；Done／藍燈的權限由 AI Hooks 與 CI 把關。
- 手機上的任何編輯、帳號管理、專案設定或 Git 寫入操作。
- Force push（含 --force-with-lease）、改寫已推送 commit、`--no-verify`：任何角色、任何平台都不提供，也沒有開關可打開。
- Codex、Gemini CLI、Copilot、Windsurf、Cline 的 hook 安裝（這些工具只裝規則與技能）。
- 外部工單工具的雙向同步（技能要求「另有工單工具時同步」，看板本身不實作同步）。
- 安裝檔程式碼簽章（目前未簽章，SmartScreen 會警告）。
- 公開網際網路直接曝露看板伺服器（任何遠端存取都必須經過驗證與加密，見第 8、10 章）。

3 系統架構

3.1 整體架構

看板採「瀏覽器 ⇄ 本機 PowerShell 伺服器 ⇄ 檔案／Git／GitHub API」三層。伺服器為 `System.Net.HttpListener`，單一執行緒事件迴圈：沒有請求時每 500 ms 檢查各專案 board.json 的 ETag、防抖自動 commit、遠端同步與 GitHub 輪詢；有請求時依路徑分派。

圖 3-1　系統架構（現況：只聽 localhost；虛線＝建議／待決）



資料流：AI 代理寫入 board.json → 伺服器偵測 ETag 變更 → SSE「board」事件 → 瀏覽器重抓 /api/p/<id>/board.json → 2 秒後自動 git commit → SSE「history」。
Token 只存在伺服器記憶體 session，不寫檔、不送到瀏覽器；伺服器重啟或登出即失效。
手機連線路徑目前不存在（HttpListener 只綁 localhost），屬第 10 章建議方案。

圖 3-1　系統架構（瀏覽器 ⇄ serve-board.ps1 HttpListener ⇄ board.json／git／GitHub API）

3.2 元件說明

元件	技術	職責
board.html	單一 HTML／原生 JS，零相依	看板｜Git 兩個分頁、Inspector、燈號篩選、通知、登入與設定畫面。EventSource 接收 SSE；沒有 SSE 時每 3 秒輪詢（`?poll=` 可調，最小 1 秒）；以 file:// 開啟時可用「載入 JSON 檔」持續監看（Chrome／Edge）。
serve-board.ps1	PowerShell 5.1／7、HttpListener	驗證、授權、靜態頁、REST API、SSE、檔案監看、自動 commit、Git 讀寫、GitHub 輪詢、遠端同步、CLI 管理指令（-AddUser、-AddProject、-SetGitHubClientId…）。
start-board.ps1／.cmd	PowerShell／批次檔	一鍵啟動（帶 -InitGit），失敗時顯示原因並等待 Enter；常見原因為 8765 埠被占用。
board.schema.json	JSON Schema 2020-12	board.json 結構與條件規則（Task 不給 SP、黃紅燈必填原因、Done⇔藍燈、Prod 必填回滾與核准人…）。
validate-board.ps1	PowerShell	與看板同一套規則的驗證器，供 AI Hooks、Git pre-commit、CI Guard 共用（另含同版本同環境不可重複等跨項規則）。
.ai-hooks／.githooks	PowerShell、Git	強制遵守第 2、3 層（見 4.15）。

元件	技術	職責
／ai-dev-guard.yml	hooks、GitHub Actions	
GitHub	OAuth Device Flow、REST API	登入（read:user repo）、PR、檢查、審查資料。

3.3 安裝根目錄檔案配置

預設安裝到 `D:\AI_INSTRUCTIONS`（母版），再複製到專案根目錄（例如 `D:\AIAD`）；也可以直接把安裝路徑選成專案資料夾。看板執行層位於安裝根目錄最外層。

```
D:\AI_INSTRUCTIONS\                ← 安裝資料夾（或專案資料夾）
├─ skills\<id>\SKILL.md            ← 母版（5 個技能）
├─ .cursor\skills\<id>\SKILL.md  .cursor\rules\<id>.mdc  .cursor\hooks.json
├─ .claude\skills\<id>\SKILL.md  .claude\settings.json   CLAUDE.md
├─ .github\copilot-instructions.md  .github\workflows\ai-dev-guard.yml
├─ .windsurf\rules\<id>.md        .clinerules\<id>.md
├─ AGENTS.md  GEMINI.md
├─ .ai-hooks\ai-hook.ps1  config.json  logs\hooks.log
├─ .githooks\pre-commit  commit-msg
├─ docs\ai-dev-skills\guide.html  BOARD-TEMPLATE.md  BRANCH-PROTECTION.md  templates\
├─ start-board.cmd  start-board.ps1  serve-board.ps1  board.html
├─ board.example.json  board.schema.json  validate-board.ps1   ← 看板執行層（最外層）
├─ .ai-board\  users.json projects.json pending.json github-oauth.json git-audit.log tmp\
│      ← 帳號與專案設定（.gitignore = *，不進 git；移除時保留）
├─ .ai-dev-skills-manifest.txt   ← 安裝清單（移除時使用）
└─ uninstall.exe                 ← 只有 exe 安裝才有
```

3.4 主要資料流

1. AI 代理修改 `docs/board.json`（AI Hooks 先用 validate-board.ps1 檢查，不合就不寫入）。
2. 伺服器偵測 ETag 改變 → SSE 推送 `board` 事件 → 瀏覽器以 If-None-Match 重抓 `/api/p/<id>/board.json`。
3. 約 2 秒防抖後自動 `git commit`（訊息如 `board: T-3(light,tdd)`）→ SSE `history` 事件。
4. GitHub 輪詢（預設 60 秒、ETag 快取）偵測新 PR、合併／關閉、CI 狀態變更 → SSE `pr` 事件 → 通知。
5. Git 寫入操作完成 → SSE `git` 事件 → 所有開著的看板自動更新；遠端同步錯誤 → SSE `sync` 事件與橫幅。

4 功能規格

每個模組依「功能描述、User Stories、BDD 驗收情境、權限、桌機 vs 手機」撰寫。User Story 編號 US-<模組>-<序號>；BDD 情境為驗收基準，對應測試命名建議 s_<模組>_<序號>_*。

4.1 AI 開發流程技能包

功能描述

五個技能把「從規劃到 PR」拆成三階段，由不同等級的模型負責，每個關卡都要有證據才放行。

- **ai-dev-pipeline（總流程）**：串起三階段與關卡；失敗經驗寫入 docs/LESSONS.md；同一工單退回兩次以上由高階模型親自診斷。
- **ai-dev-preproduction（預製期・高階）**：需求 → UI/UX（參考 2–3 個市面產品）→ 架構骨架（預設六角形＋洋蔥、五層：渲染 → API → 命令 → 應用 → 領域）→ Stories/Epics → BDD → TDD（必須有有效 RED）→ 工單 → 輸入工單工具；每步存入 docs/ 並經使用者確認。
- **工單切片（一輪原則）**：每張工單必須一輪可完成並通過 TDD、可獨立合併，不留做一半的工項。
- **ai-dev-implementation（實作期・中低階）**：一個代理一張工單、只改允許的檔案；Red → Green → Refactor；做不完回報 RESLICE_REQUIRED；不得修改 TDD，只能標 READY_FOR_TDD。
- **ai-dev-pr-acceptance（驗收期）**：A 獨立實測（操作介面、截圖錄影，平台不符記 NOT RUN）＋ B 獨立高階審查並提交 PR；部分完成、TODO 樁、TDD 失敗或 skip 一律退回。
- **ai-dev-production-readiness（上線就緒）**：測試策略（核心覆蓋率 ≥ 80%，領域層趨近 100%）、資料一致性與併發、可觀測性、Secure Coding 分層防禦、記憶體五原則、演算法與效能、發布就緒與回滾。

User Stories

ID	User Story（作為…我想要…以便…）
US-1-1	作為產品負責人，我想要用一句提示啟動總流程，以便 AI 代理從規劃一路做到 PR。
US-1-2	作為高階模型，我想要依預製期步驟產出規格、BDD、TDD 與可一輪完成的工單，以便中低階代理能準確實作。
US-1-3	作為產品負責人，我想要開發、實測、審查由三個獨立代理執行，以便驗收結果可信。

BDD 驗收情境

情境：工單過大須重切	
假設 Given	開發代理接到一張工單
當 When	代理判斷無法一輪完成並通過 TDD
那麼 Then	回報 `RESLICE_REQUIRED`，卡片退回 Backlog，由高階模型重切；不得留下半成品
情境：驗證者與開發者相同即無效	

假設 Given	一份驗收紀錄
當 When	紀錄中的開發者與驗證者身分相同
那麼 Then	驗收結果一律無效，工單不得進 Done
情境：TDD 必須先有有效 RED	
假設 Given	預製期產出 TDD
當 When	測試在實作前執行
那麼 Then	測試必須失敗（有效 RED）並保存失敗輸出，否則關卡不放行

權限

- 技能為文件，無執行期權限；角色限制由 AI Hooks、Git Hooks、CI Guard 與看板伺服器落實（見 4.15、第 7 章）。
- 只有高階模型或使用者能把工單標成完成（Done／藍燈）。

桌機 vs 手機

平台	行為
桌機（完整操作）	在 AI 工具（Cursor `@ai-dev-...`、Claude Code `/ai-dev-...`）中使用；看板顯示流程結果。
手機（監看）	不適用（技能在桌機 AI 工具執行）；手機只看到結果燈號與通知。

4.2 多 AI 工具安裝程式

功能描述

提供兩種安裝形式，內容相同，皆由 build_payload.py 產生 payload、gen_nsi.py 產生 NSIS 腳本：

- **AI-Dev-Skills-Setup.exe（NSIS）**：勾選要安裝的 AI 工具（預設全選）→ 選資料夾（預設 D:\AI_INSTRUCTIONS）→ 完成後開啟指南；建立桌面與開始功能表「AI Dev 看板」捷徑；移除用 uninstall.exe 或「設定 → 應用程式」。未簽章，SmartScreen 需按「其他資訊 → 仍要執行」。
- **AI-Dev-Skills-PowerShell.zip**：`install.ps1 -ProjectPath D:\AIAD [-Tools Cursor,Claude,…] [-NoOpen]`；`uninstall.ps1 -ProjectPath`。-Tools 可用 Cursor, Codex, Claude, Copilot, Windsurf, Gemini, Cline, Hooks, GitHooks, CIGuard；母版 skills 與指南一律安裝。
- **冪等與備份**：內容相同不重寫；不同則先備份為 .bak 再覆蓋；移除時只刪本程式安裝的檔案（依 .ai-dev-skills-manifest.txt）並還原 .bak。
- **既有 hook 設定**：PowerShell 版把本套件 hook 合併進既有 .cursor\hooks.json、.claude\settings.json（保留原設定）；exe 版備份後改寫，需手動從 .bak 合併。
- **升級遷移**：舊版看板在 docs\ai-dev-skills\board\；新版若安裝根目錄尚無 .ai-board，會把舊 .ai-board（帳號與專案設定）搬到 <安裝根>\.ai-board，並刪除舊看板執行檔。serve-board.ps1 啟動時也會做同樣搬移。

AI 工具	安裝檔案（相對安裝根）	讀取方式
（共用）母版	skills\<id>\SKILL.md	手動參考或給其他工具
Cursor	.cursor\skills\<id>\SKILL.md、.cursor\rules\<id>.mdc	Skills 依描述自動載入；Rules alwaysApply: false
Codex／通用代理	AGENTS.md	專案根目錄自動讀入（含全部技能）
Claude Code	CLAUDE.md、.claude\skills\<id>\SKILL.md	CLAUDE.md 為索引；Skills 依描述載入
GitHub Copilot	.github\copilot-instructions.md	Copilot Chat／Agent 自動套用
Windsurf	.windsurf\rules\<id>.md	trigger: model_decision
Gemini CLI	GEMINI.md	專案根目錄自動讀入
Cline／Roo Code	.clinerules\<id>.md	資料夾內規則自動套用
AI Hooks	.ai-hooks\、.cursor\hooks.json、.claude\settings.json	Cursor 與 Claude Code 動作前後自動檢查
Git Hooks／CI Guard	.githooks\、.github\workflows\ai-dev-guard.yml	commit 前與 PR 檢查

User Stories

ID	User Story（作為…我想要…以便…）
US-2-1	作為產品負責人，我想要勾選我用的 AI 工具一次安裝，以便各工具都讀到同一套流程。
US-2-2	作為產品負責人，我想要重複執行安裝來更新技能而不破壞我的既有設定，以便安全升級。
US-2-3	作為產品負責人，我想要乾淨移除並還原原本的檔案，以便不留下殘檔。
US-2-4	作為舊版使用者，我想要升級後帳號與專案設定自動搬到新位置，以便不用重新設定。

BDD 驗收情境

情境：只安裝部分工具	
假設 Given	專案資料夾 D:\AIAD
當 When	執行 `install.ps1 -ProjectPath D:\AIAD -Tools Cursor,Claude`
那麼 Then	只安裝 Cursor 與 Claude Code 檔案，加上母版 skills、指南與看板執行層；manifest 列出所有已安裝檔案
情境：既有 AGENTS.md 先備份	
假設 Given	專案已有自己的 AGENTS.md
當 When	安裝 Codex 元件
那麼 Then	舊檔備份為 AGENTS.md.bak 後安裝；移除時自動還原
情境：重複安裝不重寫相同檔案	
假設 Given	已安裝同版本
當 When	再次執行安裝
那麼 Then	內容相同的檔案不重寫、不產生新的 .bak
情境：舊版設定遷移	
假設 Given	安裝根有 docs\ai-dev-skills\board\.ai-board，且 <安裝根>\.ai-board 不存在
當 When	執行新版安裝或啟動看板
那麼 Then	設定被搬到 <安裝根>\.ai-board，舊看板執行檔刪除，使用者不需重新設定帳號

權限

- 安裝需寫入目標資料夾的檔案系統權限；不需管理員權限（安裝到使用者可寫的位置時）。**【未驗證】**exe 在受限帳號下的行為。

桌機 vs 手機

平台	行為
桌機（完整操作）	完整安裝、移除、升級（僅 Windows 桌機）。
手機（監看）	不提供。

4.3 進度看板：工作層級與欄位

功能描述

預製期一律建立 docs/board.json，以 HTML 看板顯示（Markdown 備用 docs/BOARD.md）。

- **層級**：Epic（數月～一年，不估）→ Feature（1～2 Sprint，不估）→ User Story（1 Sprint 內，Story Point）→ Task（1～2 天、一輪做完並 TDD Green，**小時，不給 SP**）→ PR（一個 Task 一個 PR，分支 feature/<task-id>-<slug>）。另有 Deployment 層級（見 4.7）。
- **欄位**：Backlog → Todo → In Progress → In Review／In QA（CI 與 TDD Green）→ Done／Shipped。
- **狀態對照**：READY_FOR_TDD = In Review；RESLICE_REQUIRED = 退回 Backlog；Done 只在獨立驗證通過且 PR 合併後。
- **卡片**只顯示精簡徽章（層級、ID、標題、燈號、BDD x/y Green、📝 備註數），細節在 Inspector。
- **BDD 規則**：Story 進 In Review／Done 時情境必須全部 Green；Task 標 TDD Green 時，涵蓋的情境（scenarioIds）也要 Green。
- **格式錯誤**（例如 Task 有 storyPoints、Story 有 hours、黃紅燈沒寫原因、Done 不是藍燈）時頁面顯示紅色橫幅，畫面保留上一版不會壞掉。

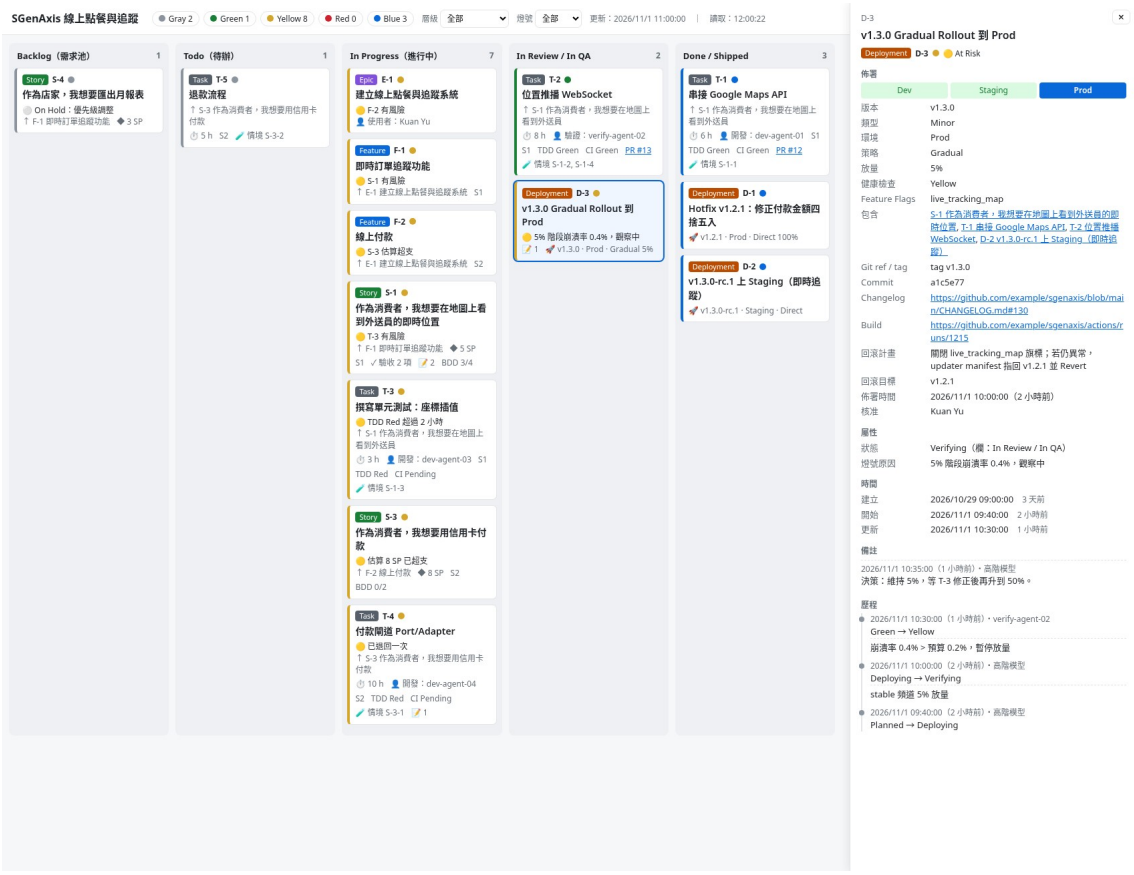


圖 4-1　桌機看板主畫面（五欄、燈號、Inspector）

User Stories

ID	User Story（作為…我想要…以便…）
US-3-1	作為產品負責人，我想要在五欄看板看到所有 Epic 到 Task，以便掌握整體進度。
US-3-2	作為高階模型，我想要 schema 擋下錯誤的估算方式，以便 SP 與工時不重複計算。
US-3-3	作為產品負責人，我想要 board.json 寫壞時看板仍顯示上一版並提示錯誤，以便不中斷監看。

BDD 驗收情境

情境：Task 不得有 Story Point	
假設 Given	board.json 中某 Task 的 estimate 含 storyPoints
當 When	看板載入此檔
那麼 Then	顯示紅色錯誤橫幅並指出該項；畫面維持上一版有效內容
情境：Story 情境未全 Green 不得完成	
假設 Given	Story S-1 有一個情境狀態為 Red
當 When	代理把 S-1 設為 Done
那麼 Then	驗證失敗（看板紅色橫幅；AI Hooks／pre-commit 阻擋寫入或提交）
情境：欄位顯示	
假設 Given	board.json 有效
當 When	使用者開啟看板

那麼 Then	依 status 分到 Backlog／Todo／In Progress／In Review／Done 五欄，卡片顯示徽章

權限

- 所有已授權該專案的角色皆可檢視；看板沒有線上編輯。

桌機 vs 手機

平台	行為
桌機（完整操作）	五欄橫向看板、卡片可用 Tab 選取、Enter 開 Inspector；網址 `board.html#S-1` 直接開卡；附圖 4-1。
手機（監看）	**【建議】**不顯示五欄；改為「燈號摘要＋需注意清單（紅 → 黃）＋各欄數量條」，點卡片開唯讀詳細（見第 10 章）。

4.4 健康燈號與燈號篩選

功能描述

每張卡必填 light；黃、紅燈必填 lightReason（最多 200 字）。

- ⚪ **Gray**＝Not Started／On Hold：Backlog、Todo 預設；暫停時任何欄位可用。
- 🟢 **Green**＝On Track：In Progress、In Review／In QA 順利進行。
- 🟡 **Yellow**＝At Risk：工時超過估算、TDD Red 過久、審查退回一次（必填原因）。
- 🔴 **Red**＝Blocked：相依卡住、CI 失敗、待重切、驗證多次退回（必填原因）；佈署 Rolled Back 必為紅。
- 🔵 **Blue**＝Complete／Shipped：只在 Done（或 Released），且 PR 合併後由高階模型或使用者設定；Done／Released ⇔ Blue 雙向強制。
- **上層匯總**：子項全藍 → 藍；全未開始 → 灰；其餘取進行中子項最差燈號（紅 > 黃 > 綠）；不一致時在上層卡片標出。
- **燈號篩選**：頂端五個燈號按鈕顯示各燈號數量，可多選，「清除」還原；只顯示符合卡片，上層卡片因子項符合而顯示時變淡；篩選依專案記在瀏覽器，即時更新時保留。

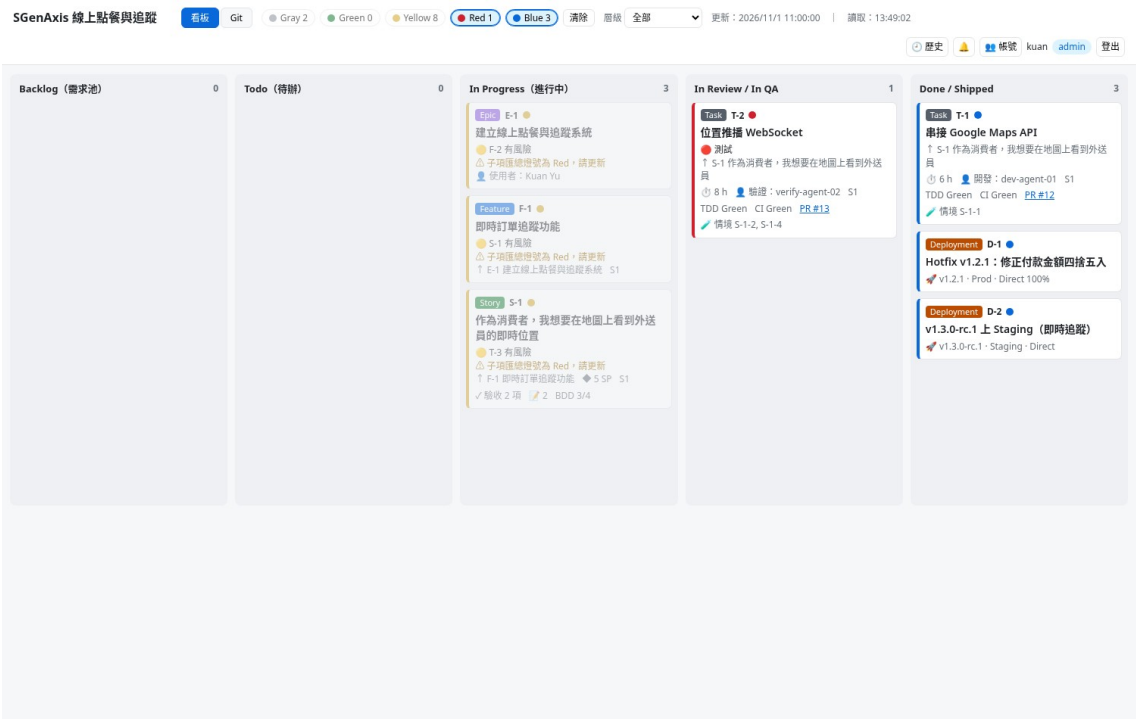


圖 4-2　燈號篩選（多選、上層變淡）

User Stories

ID	User Story（作為…我想要…以便…）
US-4-1	作為產品負責人，我想要一眼看出哪些卡片卡住或有風險，以及原因，以便優先處理。
US-4-2	作為產品負責人，我想要只看紅燈與黃燈卡片，以便聚焦風險。
US-4-3	作為產品負責人，我想要上層燈號與子項不一致時被提醒，以便發現代理漏更新。

BDD 驗收情境

情境：黃紅燈必填原因	
假設 Given	某 Task light 為 Red 且沒有 lightReason
當 When	看板或 validate-board 檢查
那麼 Then	判定格式錯誤（紅色橫幅；Hooks 阻擋寫入）
情境：多選篩選並保留	
假設 Given	專案 A 有 2 張紅、3 張黃、其他燈號若干
當 When	使用者點選 Red 與 Yellow 篩選，接著 board.json 更新
那麼 Then	只顯示紅、黃卡片（上層卡片變淡）；即時更新後篩選仍保留；切到專案 B 使用 B 自己的篩選
情境：上層匯總不一致	
假設 Given	Feature F-1 標 Green，但子 Story S-1 為 Red
當 When	看板渲染
那麼 Then	F-1 卡片標示匯總不一致
情境：開發代理不得設藍燈	

假設 Given	AI Hooks 啟用，角色 dev-agent
當 When	代理把 Task 設 light: Blue、status: Done
那麼 Then	AI Hooks 阻擋寫入 board.json

權限

- 檢視：所有已授權角色。設定藍燈／Done：僅高階模型或使用者（由 Hooks／CI 把關，看板不提供編輯）。

桌機 vs 手機

平台	行為
桌機（完整操作）	頂端燈號篩選列（圖 4-2），篩選結果即時套用於五欄。
手機（監看）	總覽頁頂端五個燈號計數；點燈號展開該燈號卡片清單（唯讀）。燈號除顏色外必附文字與數字（無障礙）。

4.5 即時更新（JSON 監看 → SSE）

功能描述

伺服器監看各專案 board.json，改變時以 Server-Sent Events 推送，頁面自動更新。

- `GET /events`：text/event-stream，`retry: 3000`；每 15 秒左右送 `: ping` 保活；連線上限 20（-MaxClients），超過回 503 E_BUSY。
- 事件種類：`board`（board.json 變更）、`history`（自動 commit 完成）、`git`（Git 操作完成）、`pr`（PR／CI 變更）、`sync`（遠端同步狀態）。事件只送給有該專案權限的連線。
- `GET /api/p/<id>/board.json` 支援 ETag／If-None-Match（304）。
- 降級：瀏覽器無 EventSource 時每 3 秒輪詢；無 PowerShell 時可用 `python -m http.server` 或 `npx http-server`；以 file:// 開啟時按「載入 JSON 檔」持續監看（Chrome／Edge）。

User Stories

ID	User Story（作為…我想要…以便…）
US-5-1	作為產品負責人，我想要 AI 代理更新 board.json 後看板自動刷新，以便不用手動重新整理。
US-5-2	作為產品負責人，我想要沒有伺服器時也能用雙擊 HTML 的方式看看板，以便離線檢視。

BDD 驗收情境

情境：檔案變更即時推送	
假設 Given	使用者已開啟專案 A 看板並連上 /events
當 When	代理儲存 A 的 board.json
那麼 Then	約 0.5 秒內收到 board 事件，頁面重抓並更新；Inspector 依 ID 保持選取

情境：無權限不收事件	
假設 Given	使用者只被授權專案 A
當 When	專案 B 的 board.json 變更
那麼 Then	該使用者的 SSE 連線不會收到 B 的事件
情境：連線數上限	
假設 Given	已有 20 個 SSE 連線
當 When	第 21 個連線要求 /events
那麼 Then	回 503 E_BUSY；頁面改用輪詢

權限

- 需登入（-NoAuth 除外）；事件依專案授權過濾。

桌機 vs 手機

平台	行為
桌機（完整操作）	SSE 常駐；分頁背景時仍接收。
手機（監看）	【建議】頁面可見時才建立 SSE，切到背景即關閉，回到前景時以 ETag 重抓一次（省電、避免占用 20 個連線上限）；推播改由 Web Push 負責。

4.6 Inspector、時間戳、歷程與備註

功能描述

點卡片（或 Tab 選取後按 Enter）在右側開啟 Inspector；× 或 Esc 關閉。

- 內容：層級、ID、標題、上層路徑（可點）、子項、狀態、燈號與原因、估算、負責角色、Sprint、TDD、CI、PR、驗收標準、完整 BDD 情境與測試、備註、時間與歷程、提到此卡的 commit。
- **時間戳**：createdAt、updatedAt、startedAt、completedAt，ISO 8601 含時區（如 2026-11-01T09:30:00+08:00）；顯示本地時間與「3 小時前」相對時間；completedAt 不可早於 startedAt。
- **歷程**：history: [{at, from, to, by, note}]（最多 200 筆）；代理每次移動卡片或改燈號都更新 updatedAt 並追加一筆；Inspector 以時間軸顯示（新的在上）。
- **備註**：notes: [{at, by, text}]（最多 100 筆、每則 2000 字）與單一 remark；保留換行，**一律當純文字顯示**（防 XSS）。
- 即時更新時依 ID 保持選取並刷新；項目被刪除時顯示「已移除」。
- 【待決】completedAt 由誰設定（建議：高階模型或使用者，與藍燈權限一致，見第 14 章）。
- 【未驗證】已知問題：Inspector 開啟時可能覆蓋頁首右側按鈕（見第 13 章）。

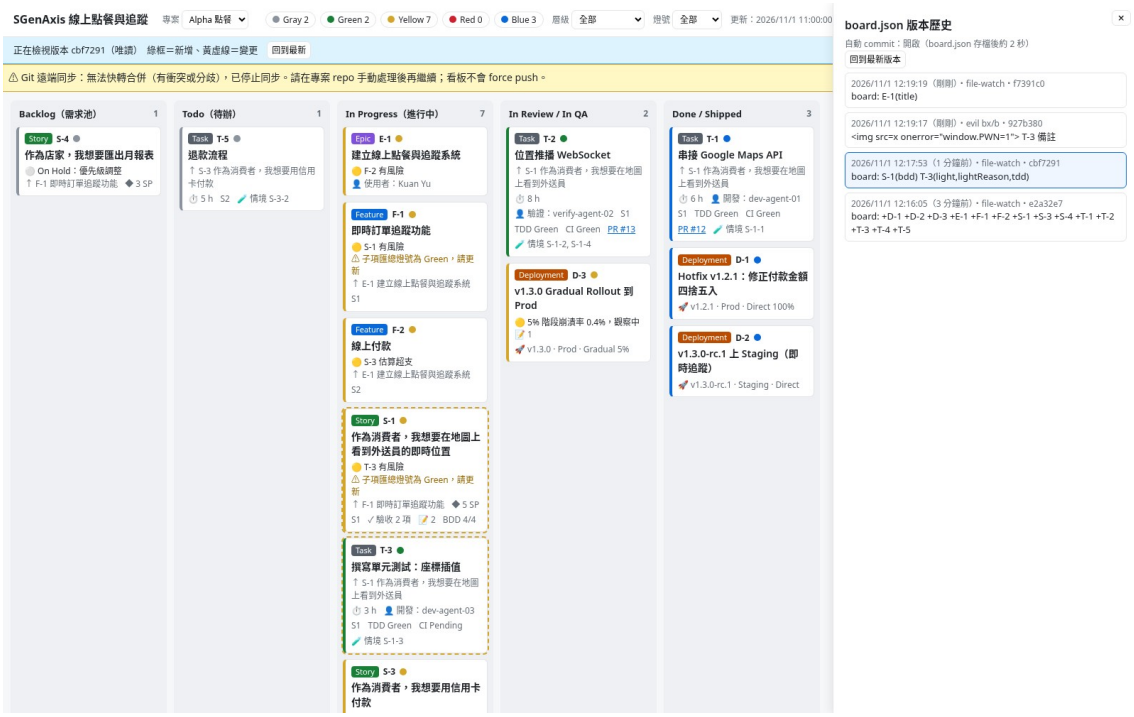


圖 4-3　版本歷史與 Inspector 時間軸

User Stories

ID	User Story（作為…我想要…以便…）
US-6-1	作為產品負責人，我想要點一張卡就看到完整細節與時間軸，以便了解它怎麼走到現在。
US-6-2	作為高階模型，我想要在卡片留下決策與阻擋紀錄，以便後續代理延續脈絡。
US-6-3	作為產品負責人，我想要用網址直接分享某張卡，以便溝通。

BDD 驗收情境

情境：深連結開卡	
假設 Given	使用者已登入且有專案 sgenaxis 權限
當 When	開啟 `/?p=sgenaxis#S-1`
那麼 Then	切到 sgenaxis 並開啟 S-1 的 Inspector
情境：卡片被刪除	
假設 Given	Inspector 正顯示 T-9
當 When	board.json 更新後 T-9 不存在
那麼 Then	Inspector 顯示「已移除」，不報錯
情境：備註純文字	
假設 Given	某備註含 `<script>` 字樣
當 When	Inspector 顯示該備註
那麼 Then	以純文字顯示，不執行任何腳本
情境：完成時間不得早於開始	

假設 Given	某卡 completedAt 早於 startedAt
當 When	驗證 board.json
那麼 Then	判定格式錯誤

權限

- 所有已授權角色可檢視；備註與歷程由代理寫入 board.json（看板無線上編輯）。

桌機 vs 手機

平台	行為
桌機（完整操作）	右側固定側欄（圖 4-3），可與看板同時檢視。
手機（監看）	【建議】全螢幕唯讀詳細頁，欄位分組可摺疊（狀態／估算／BDD／歷程／備註），歷程預設顯示最近 5 筆。

4.7 佈署與版本卡（Deployment）

功能描述

level = Deployment 的卡片記錄每次佈署，與一般卡共用看板。

- **欄位**：version（SemVer，可含 -alpha/-beta/-rc.N）、releaseType（Major/Minor/Patch/Hotfix/RC）、environment（Dev/Staging/Prod）、strategy（Direct/Canary/Gradual/FeatureFlag）、rolloutPercent、featureFlags、includes、gitRef、commitSha、changelogUrl、buildUrl、healthChecks、rollbackPlan、rollbackRef、deployedAt、approvedBy。
- **狀態**：Planned（Todo 欄）→ Deploying（In Progress）→ Verifying（In Review，健康 Green）→ Released／Rolled Back（Done 欄）。Released＝藍、Rolled Back＝紅、Canary／放量有風險＝黃。
- **規則**：Prod 必填 rollbackPlan 與 approvedBy（不可以是開發代理）；Hotfix 必須是 Patch（版本最後一碼 ≥ 1）；同版本同環境不可重複；releaseType 要和前一版的升版相符。
- **流程**：PR Green → Squash and Merge 進 main → vX.Y.Z-rc.1 上 Staging → 獨立驗證 → 使用者或高階模型核准 → vX.Y.Z 上 Prod，5% Gradual Rollout → 健康 Green 且 100% → Released。出事：先關 Feature Flag，再指回 rollbackRef 並 git revert（不 force push），卡片設 Rolled Back。
- **Hotfix**：從 tag 拉 hotfix/<version>；仍要 TDD Green＋獨立驗證；上線後 back-merge 回 main。
- Inspector 顯示 Dev → Staging → Prod 進度條與所有佈署欄位。
- 【待決】版本唯一性是「每個環境」還是「整個看板」（現況實作為 version@environment 唯一）。

User Stories

ID	User Story（作為…我想要…以便…）
US-7-1	作為產品負責人，我想要看到某版本目前在哪個環境、放量多少、健康如何，以便決定是否全量。

ID	User Story（作為…我想要…以便…）
US-7-2	作為產品負責人，我想要 Prod 佈署一定有回滾計畫與非開發代理的核准人，以便降低風險。

BDD 驗收情境

情境：Prod 必填回滾與核准	
假設 Given	一張 Deployment 卡 environment = Prod
當 When	缺 rollbackPlan 或 approvedBy
那麼 Then	驗證失敗
情境：核准人不可是開發代理	
假設 Given	Prod 佈署卡
當 When	approvedBy 為 "dev-agent"
那麼 Then	驗證失敗（schema 以 (?i)dev[-_]?agent 排除）
情境：Hotfix 必須是 Patch	
假設 Given	releaseType = Hotfix
當 When	version 為 v1.4.0
那麼 Then	驗證失敗；v1.3.1 才合法
情境：同版本同環境重複	
假設 Given	已有 v1.3.0@Staging
當 When	再新增 v1.3.0@Staging
那麼 Then	驗證失敗；v1.3.0@Prod 合法

權限

- 設定 Released／approvedBy：高階模型或使用者（AI Hooks 阻擋開發代理設定 Done／藍燈／Released／approvedBy）。

桌機 vs 手機

平台	行為
桌機（完整操作）	Inspector 顯示環境進度條、所有欄位與連結。
手機（監看）	【建議】總覽頁「最新佈署」區塊：每個環境最新版本、狀態燈號、放量 %、健康檢查；不可核准或回滾。

4.8 帳號、登入與授權

功能描述

- **GitHub 登入（主要）**：OAuth Device Flow，範圍 `read:user repo`。頁面顯示代碼 → 使用者到 github.com/login/device 輸入並授權。**第一個登入的 GitHub 帳號成為管理者（只能在本機）**。
- **待核准**：其他人用 GitHub 登入後進入「待核准」清單；管理者在「👥 帳號」選角色與可看的專案後核准或拒絕；可改角色、改專案授權、移除帳號（至少保留一位管理者，E_LAST_ADMIN）。
- **本機帳號（離線備用）**：登入頁下方「本機帳號登入」；密碼至少 12 字，PBKDF2-SHA256 60 萬次、16 byte 隨機鹽，存 .ai-board/users.json；同帳號連錯 5 次鎖 15 分鐘；錯誤只回錯誤碼。
- **Session**：Cookie `aib_sid` 為 32 byte 隨機值、HttpOnly、SameSite=Strict，伺服器只存 SHA-256；閒置 8 小時失效；最多 100 個 session。
- **GitHub token**：只存在伺服器記憶體 session，不寫檔、不送到瀏覽器；重啟或登出即失效。
- **角色**：viewer、dev-agent、verifier、high-tier、admin；每個帳號只看得到被授權的專案（admin 看全部）；不存在與無權限的專案一律回 403（不洩漏存在與否）。
- **-NoAuth**：單人使用可關閉登入（畫面顯示警告）。CLI：`-AddUser`、`-RemoveUser`、`-ResetPassword`、`-GrantUser 名稱 -Project id -Role 角色`。



圖 4-4　登入畫面（GitHub 登入為主、本機帳號備用）

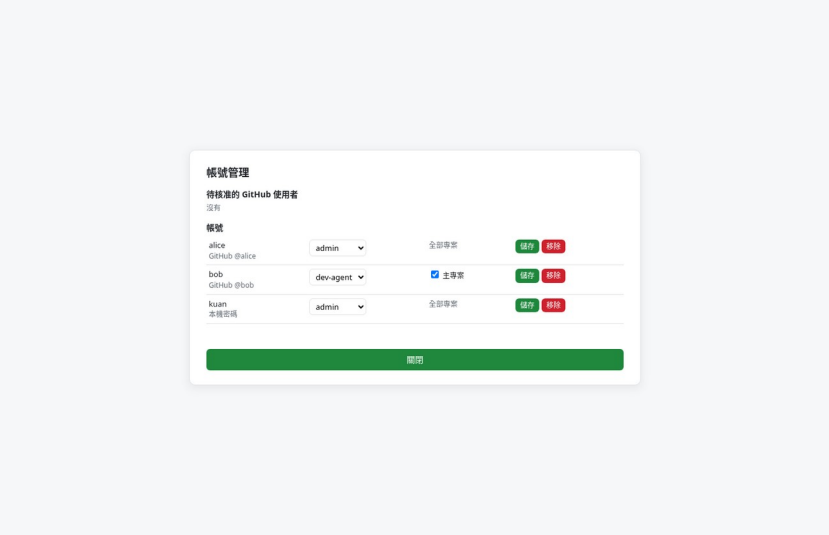


圖 4-5　帳號管理（待核准、角色、專案授權）

User Stories

ID	User Story（作為…我想要…以便…）
US-8-1	作為產品負責人，我想要用 GitHub 帳號登入而不必管理密碼，以便安全又方便。
US-8-2	作為管理者，我想要核准新使用者並指定角色與專案，以便控制誰能看什麼。
US-8-3	作為管理者，我想要 GitHub 連不上時仍能用本機帳號登入，以便不被鎖在外面。

BDD 驗收情境

情境：第一位 GitHub 使用者成為管理者	
假設 Given	已設定 Client ID，尚無任何帳號
當 When	使用者在本機完成 Device Flow 登入
那麼 Then	該帳號成為 admin
情境：其他人進入待核准	
假設 Given	已有管理者
當 When	另一個 GitHub 帳號完成登入
那麼 Then	該帳號進入待核准清單，無法看到任何專案（E_PENDING），直到管理者核准
情境：不能移除最後一位管理者	
假設 Given	系統只有一位 admin
當 When	管理者嘗試移除自己或降級
那麼 Then	拒絕（E_LAST_ADMIN／E_SELF）
情境：連錯鎖定	
假設 Given	本機帳號 kuan
當 When	連續輸入錯誤密碼 5 次
那麼 Then	帳號鎖定 15 分鐘（E_LOCKED），回應只有錯誤碼
情境：BOLA：不能看未授權專案	

假設 Given	使用者只被授權專案 A
當 When	請求 `/api/p/B/board.json`
那麼 Then	回 403 E_FORBIDDEN，不含任何資料

權限

- 帳號管理、核准、Client ID 設定：admin。登入：所有人。

桌機 vs 手機

平台	行為
桌機（完整操作）	完整登入（GitHub／本機）、帳號管理畫面（圖 4-4、4-5）。
手機（監看）	【建議】可用 GitHub Device Flow 登入（手機開 github.com/login/device 輸入代碼即可）；**手機 session 由伺服器標記為唯讀**；不提供帳號管理；本機帳號是否允許在手機登入【待決】。

4.9 首次啟動與新增專案

功能描述

- **先啟動，再設定**：安裝後雙擊「AI Dev 看板」捷徑（或 start-board.cmd）→ 伺服器在 http://localhost:8765/ 啟動並自動開瀏覽器（帶 -InitGit）；黑色視窗要保持開著。
- **第一次使用：設定登入**（圖 4-6）：① 設定 GitHub 登入：GitHub → Settings → Developer settings → OAuth Apps → New OAuth App，Homepage 與 callback 填 http://localhost:8765/，勾 Enable Device Flow，把 Client ID 貼到設定頁（不需 secret）；或 `-SetGitHubClientId`。② 建立本機管理員帳號（備用）。
- `/api/setup` 只在尚未設定時可用（已設定回 E_ALREADY_SETUP），且只接受本機請求。
- **新增專案**：管理者登入後若沒有專案，看到「新增專案」：專案 ID（`^[a-z0-9][a-z0-9-]{0,39}$`）、名稱、board.json 路徑（預設 <安裝根>\docs\board.json）、選填 GitHub repo（owner/repo）。board.json 不存在時由範例建立並自動 git commit。
- CLI：`-AddProject id -Name -BoardPath -GitHubRepo [-RemoteSync]`、`-RemoveProject`、`-ListProjects`。



圖 4-6　第一次使用：設定登入

User Stories

ID	User Story（作為…我想要…以便…）
US-9-1	作為第一次使用的產品負責人，我想要不打指令就在網頁上完成設定，以便快速開始。
US-9-2	作為管理者，我想要在網頁新增專案並自動建立 board.json，以便立刻看到看板。

BDD 驗收情境

情境：首次設定	
假設 Given	全新安裝，.ai-board 無任何設定
當 When	開啟 http://localhost:8765/
那麼 Then	顯示「第一次使用：設定登入」，可設定 Client ID 或建立本機管理員
情境：8765 被占用	
假設 Given	另一個看板已在執行
當 When	再次啟動看板
那麼 Then	視窗顯示「port 8765 已被占用」並等待 Enter
情境：新增專案自動建檔	
假設 Given	管理者登入、board.json 路徑不存在
當 When	送出新增專案
那麼 Then	由範例建立 board.json、自動 git commit、看板載入
情境：非本機不可做首次設定	
假設 Given	【建議】啟用 LAN 綁定後
當 When	手機嘗試開啟首次設定
那麼 Then	拒絕；首次設定只能在本機桌機完成

權限

- 首次設定：本機；新增專案：admin。

桌機 vs 手機

平台	行為
桌機（完整操作）	完整首次設定與新增專案流程（圖 4-6）。
手機（監看）	不提供；手機若遇到未設定狀態，顯示「請在桌機完成設定」。

4.10 多專案切換

功能描述

- 頂端下拉選單切換專案，每個專案各有自己的 board.json 與 Git repo；最多 50 個專案。
- 網址 ?p=專案#卡片 可直接分享；其他專案的新通知在選單上顯示未讀數。
- 每個專案獨立：檔案監看、自動 commit、遠端同步（選用）、GitHub PR 通知、燈號篩選記憶。
- 專案只用 ID 指定，伺服器**從不接受用戶端傳來的路徑**。

User Stories

ID	User Story（作為…我想要…以便…）
US-10-1	作為產品負責人，我想要在多個產品之間切換看板，以便同時管理多條產品線。
US-10-2	作為產品負責人，我想要看到其他專案的未讀通知數，以便不漏掉事件。

BDD 驗收情境

情境：切換專案	
假設 Given	使用者有專案 A、B 權限
當 When	在下拉選單選 B
那麼 Then	載入 B 的 board.json 與 B 的篩選設定；網址更新為 ?p=B
情境：其他專案未讀	
假設 Given	目前在 A
當 When	B 有新 PR 通知
那麼 Then	選單上 B 顯示未讀數
情境：不洩漏專案存在	
假設 Given	使用者沒有 C 的權限
當 When	請求 /api/p/C/…（C 存在或不存在）
那麼 Then	一律回 403 E_FORBIDDEN

權限

- 可見專案依授權；admin 看全部。

桌機 vs 手機

平台	行為
桌機（完整操作）	下拉選單＋未讀數。
手機（監看）	【建議】總覽頁頂端專案切換（同一授權規則），並提供「所有專案燈號總覽」清單（每專案一列：五燈號計數＋最新事件）。

4.11 board.json 版本歷史（Git 自動 commit）

功能描述

- board.json 存檔約 2 秒（-DebounceMs）後自動 commit 到該專案 repo（訊息如 `board: T-3(light,tdd)`）。
- 作者：最近操作、且對該專案有編輯權限的 GitHub 使用者（名字 <id+login@users.noreply.github.com>），沒有就用 file-watch。
- 「🕘 歷史」看每一版、點一版唯讀檢視並標出新增／變更；Inspector 列出提到該卡的 commit。
- API：`/history?limit=1..100&skip=&id=`、`/history/<sha>`。不在 repo 時可加 -InitGit；沒有 git 時歷史功能停用（503 E_GIT_DISABLED）。
- **遠端同步（選用，-RemoteSync）**：定期 `git pull --ff-only`，自動 commit 後 push；不能快轉或 push 被拒時停下並顯示橫幅，**絕不 force push**。

User Stories

ID	User Story（作為…我想要…以便…）
US-11-1	作為產品負責人，我想要每次 board.json 變更都自動留版，以便追查誰在何時改了什麼。
US-11-2	作為產品負責人，我想要比較某一版與前一版的差異，以便了解變化。

BDD 驗收情境

情境：防抖自動 commit	
假設 Given	專案在 git repo 內、AutoCommit on
當 When	代理在 1 秒內連續存檔 3 次
那麼 Then	最後一次存檔約 2 秒後只產生 1 個 commit，並推送 history 事件
情境：遠端分岔時停止	
假設 Given	啟用 RemoteSync，遠端有本機沒有的 commit 且不能快轉
當 When	同步週期執行
那麼 Then	停止同步、顯示橫幅（E_SYNC），不 force push

權限

- 檢視：所有已授權角色。

桌機 vs 手機

平台	行為
桌機（完整操作）	歷史清單與唯讀比對（圖 4-3）。
手機（監看）	【建議】唯讀歷史清單（最近 30 筆）；不提供版本比對的大型 diff 畫面。

4.12 PR／CI 通知

功能描述

- 伺服器以登入者的 GitHub token（沒有時用 GITHUB_TOKEN 或 gh CLI）輪詢專案 repo，預設 60 秒（-GitHubInterval），結果快取 60 秒並用 ETag。
- 事件：新 PR、PR 合併／關閉、CI 狀態變更；board.json 新增 PR 連結也會通知。
- 右上角跳出通知，點了打開對應 Task 卡（**依分支 feature/<task-id>-* 或 prUrl 對應**），否則開 PR 頁；🔔 保留最近 100 筆；可開啟瀏覽器桌面通知（頁面隱藏時才跳）。
- 未驗證的 GitHub 呼叫每小時 60 次上限，啟動時會提示驗證方式。

User Stories

ID	User Story（作為…我想要…以便…）
US-12-1	作為產品負責人，我想要 PR 合併或 CI 失敗時立即收到通知，以便快速反應。
US-12-2	作為產品負責人，我想要點通知直接打開對應的 Task 卡，以便看到上下文。

BDD 驗收情境

情境：分支對應卡片	
假設 Given	PR 分支為 feature/T-4-webhook，board.json 有 T-4
當 When	CI 由 Green 變 Red
那麼 Then	跳出通知；點擊開啟 T-4 Inspector
情境：無對應卡片	
假設 Given	PR 分支不符合 feature/<id>-* 且無 prUrl 對應
當 When	PR 合併
那麼 Then	通知點擊後開啟 GitHub PR 頁
情境：通知保留上限	
假設 Given	🔔 已有 100 筆
當 When	新通知到達
那麼 Then	保留最新 100 筆

權限

- 所有已授權該專案的角色都收到該專案通知。

桌機 vs 手機

平台	行為
桌機（完整操作）	浮動通知＋🔔 清單＋瀏覽器桌面通知。
手機（監看）	**手機核心功能**：頁面內 🔔 清單與未讀數；**【建議】**Web Push 推播（需 HTTPS 與 Service Worker），點擊開唯讀卡片。

4.13 Git 分頁：瀏覽

功能描述

- 頂端「看板｜Git」切換，針對目前專案的 repo。
- **概覽**：目前分支與 HEAD、上游領先／落後、未提交變更、stash 數、最後 fetch、remote 網址（去除帳密）、標籤（綠＝最新正式 SemVer、黃＝最新 RC）、30 天貢獻者與每日活動圖。
- **線圖**：每條分支一種顏色，畫出分岔與合併；分支／遠端／標籤／HEAD 標籤、訊息、作者頭像、時間、短 SHA；訊息含卡片 ID 或分支為 feature/<id>-* 時出現卡片徽章（含燈號）；PR 徽章；工具列篩選（所有／目前分支、遠端、標籤、分支、作者）；搜尋（SHA、訊息、作者，不符的列變淡）；捲動每次再載 200 筆。
- **commit 詳細**：完整訊息、作者與提交者、上層 commit、指向的分支／標籤、包含它的分支、相關卡片、變更檔案（A/M/D/R、+/−）與單檔 diff（上限 1 MB，超過截斷）。
- **分支**：最後 commit、與主線的領先／落後、已合併、上游、對應卡片與燈號、對應 PR 與 CI；超過 14 天標「閒置」。
- **Pull Requests**：開啟／合併／關閉、草稿、標籤、CI 與每個檢查項目、可否合併、審查者與結果、對應卡片。
- 沒有 git、remote 或 GitHub repo 時顯示說明而非錯誤。

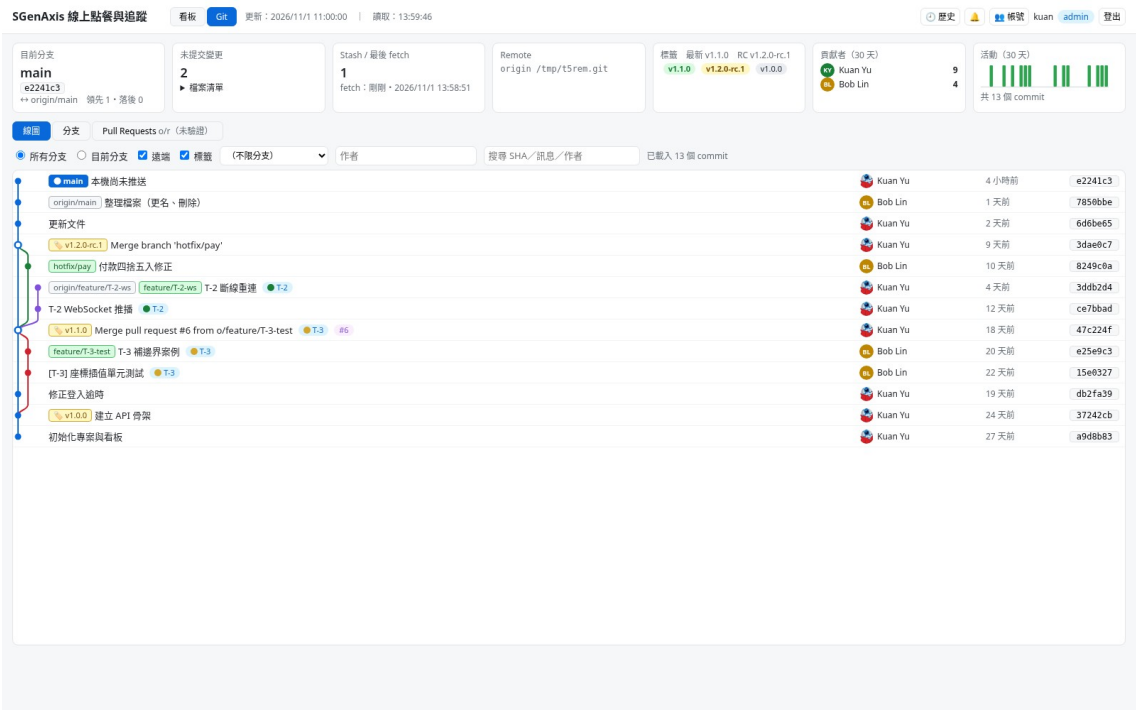


圖 4-7　Git 線圖（分支顏色、卡片與 PR 徽章）

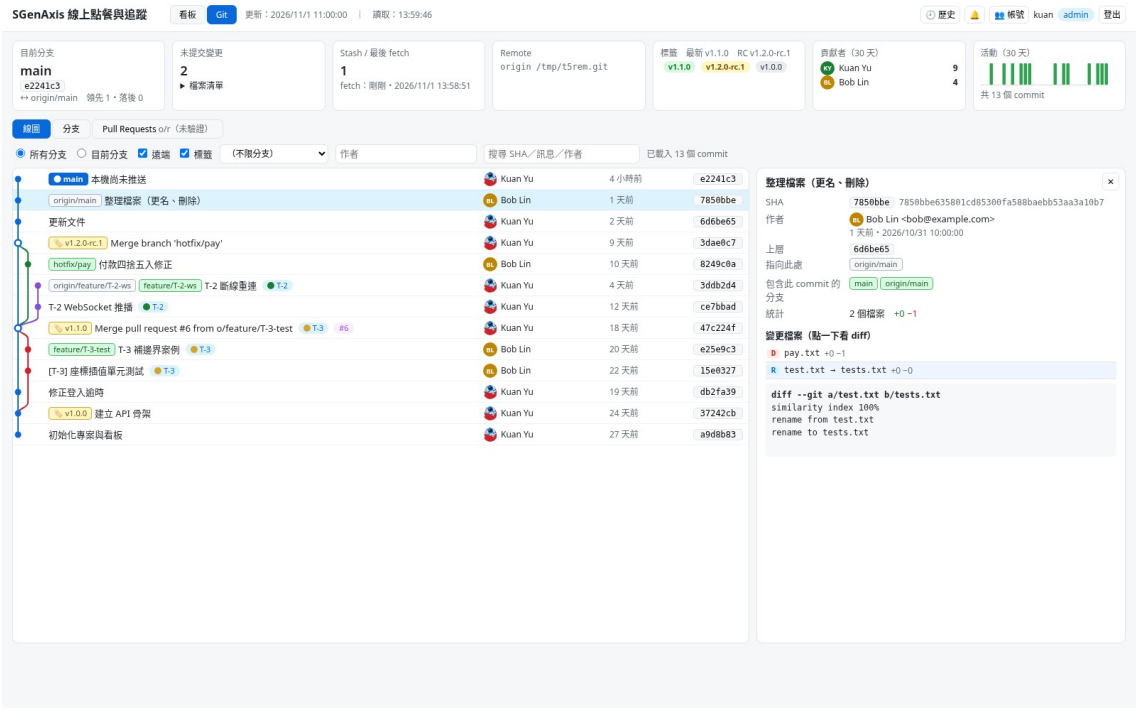


圖 4-8　commit 詳細與檔案 diff

User Stories

ID	User Story（作為…我想要…以便…）
US-13-1	作為產品負責人，我想要看到像 GitKraken 一樣的版控線圖並連到看板卡片，以便把程式變更和工單對起來。
US-13-2	作為審查代理，我想要在 PR 清單看到 CI 與審查結果，以便判斷能否合併。

BDD 驗收情境

情境：卡片徽章	
假設 Given	commit 訊息含 [T-12]
當 When	開啟線圖
那麼 Then	該列顯示 T-12 卡片徽章與燈號，點擊跳到看板卡片
情境：搜尋不斷線	
假設 Given	線圖已載入
當 When	以作者搜尋
那麼 Then	不符合的列變淡，線圖保持連續
情境：無 GitHub repo	
假設 Given	專案未設定 GitHub repo
當 When	開啟 Pull Requests
那麼 Then	顯示設定說明，而非錯誤

權限

- 所有已授權角色可瀏覽（viewer 亦可）。

桌機 vs 手機

平台	行為
桌機（完整操作）	完整 Git 分頁（圖 4-7、4-8）。
手機（監看）	【建議】只提供「PR／CI」頁（PR 清單、CI 結果、審查結果）與「分支摘要」（名稱、燈號、領先／落後）；不提供線圖與 diff。

4.14 Git 分頁：操作（僅桌機）

功能描述

- 概覽下方工具列顯示目前分支與上游（↑ 未推送／↓ 未拉取），以及 Fetch、Pull、Push、新分支、Stash；子分頁「變更」與「操作紀錄」。每個操作完成後所有開著的看板自動更新。
- **不可關閉的安全規則（任何角色）**：永遠不 force push（含 --force-with-lease，推送 refspec 不帶 +）；已推送 commit 不能 amend／rebase／reset；沒有 reset --hard（只有「全部捨棄」會用，且須輸入分支名確認）；永遠不用 --no-verify；主線、受保護分支與目前分支不能刪除。
- **每個寫入操作**：檢查 CSRF（Origin 必須是本機頁面），且【現況】`/git/op` 只接受本機（loopback）請求，非本機回 403 E_NOT_LOCAL；一次只執行一個、有 index.lock 時拒絕（E_BUSY／E_LOCKED）；驗證分支名、commit 與路徑並用 -- 分隔防參數注入；逾時（一般 2 分鐘、網路 5 分鐘，E_TIMEOUT）；失敗時顯示 git 錯誤訊息；merge／rebase 進行中禁止其他狀態變更（E_IN_PROGRESS）。

- **commit 作者**：登入的 GitHub 帳號（noreply 信箱）；本機帳號用 帳號@ai-board.local。
- **操作紀錄（稽核）**：每個寫入操作（成功或失敗）記錄時間、使用者、GitHub 帳號、操作、參數與結果於 .ai-board/git-audit.log；畫面顯示此專案最近 200 筆。

類別	操作（/git/op 的 op）	規則重點
變更	stage、unstage、stageHunk、discard、discardAll、commit（含 amend）	discard／discardAll 須輸入檔名或分支名確認，只影響本機；amend 只限未推送；commit 一定跑 hooks，有 commit-msg hook 時先檢查 Conventional Commits（E_COMMIT_FORMAT）
Stash	stashPush（可含未追蹤）、stashApply、stashPop、stashDrop	刪除須輸入 stash@{n} 確認
遠端	fetch（--prune）、pull（ff-only 預設／merge／rebase）、push	分岔時說明原因（E_NOT_FF_PULL）；push 無上游時自動設定；被拒回 E_NON_FF／E_PUSH_REJECTED；mirror remote 拒絕（E_MIRROR）
分支	branchCreate（可由卡片產生 feature/<任務 ID>-說明）、switch、branchRename、branchDelete、merge（ff／no-ff／squash／ff-only）	有未提交變更時切換先擋（E_DIRTY，可選先 stash）；未合併分支刪除須輸入名稱；主線、受保護分支（main／master／develop／trunk／release/*）、目前分支不能刪除或改名
commit 詳細	cherryPick、revert、tag、tagPush、reset（--soft／--mixed）、從此 commit 建分支、從此開始互動式 rebase	tag 必須是 SemVer（E_BAD_TAG）；reset 只限未推送 commit（E_PUSHED_REWRITE）
Rebase	rebase、rebaseInteractive（pick／reword／squash／fixup／drop，可拖曳排序）	範圍內有已推送 commit 時整個不能開始（E_PUSHED_REWRITE）
衝突	resolve、continue、skip（rebase／cherry-pick）、abort	頂端黃色橫幅；三欄合併編輯器（ours／base／theirs）；結果仍有衝突標記時擋下（E_MARKERS），除非勾「仍要標記完成」；rebase 時提醒 ours＝目標分支

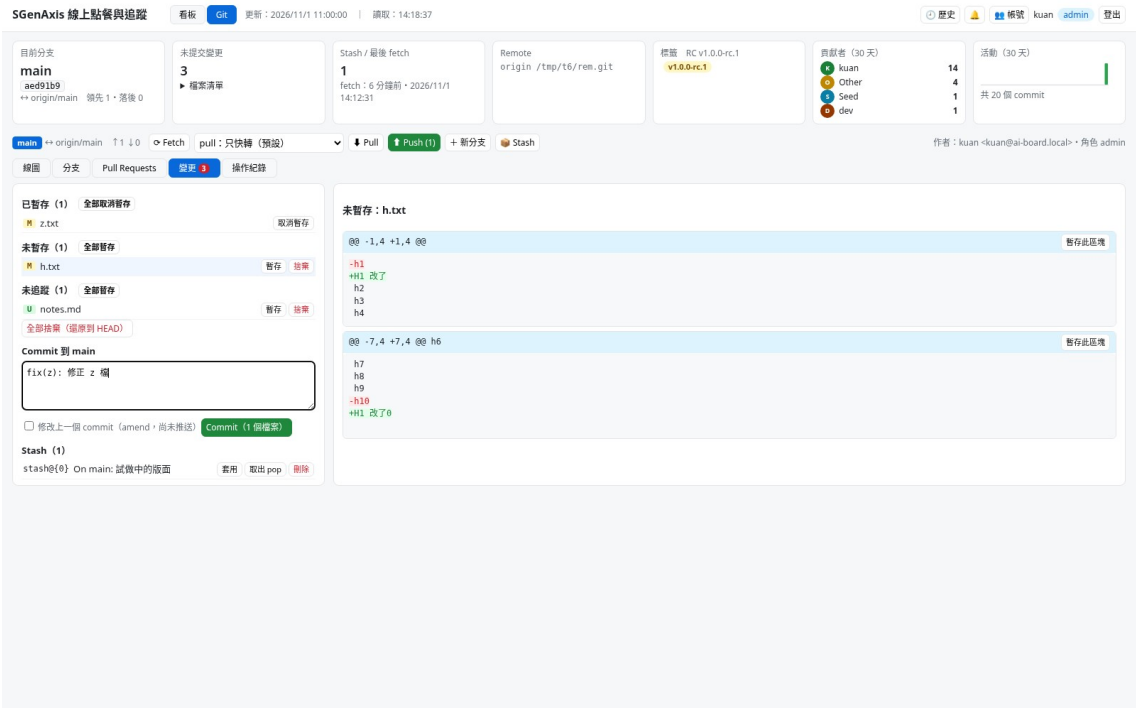


圖 4-9　Git 操作：變更、暫存、commit、工具列

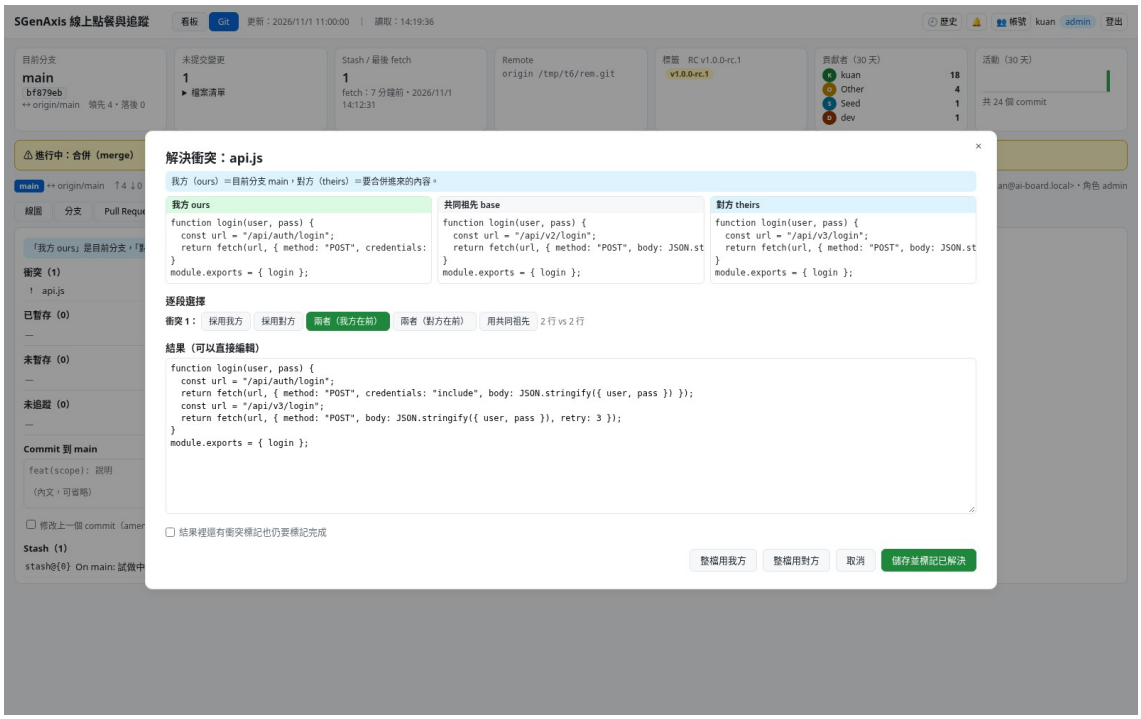


圖 4-10　三欄合併編輯器（ours／base／theirs）

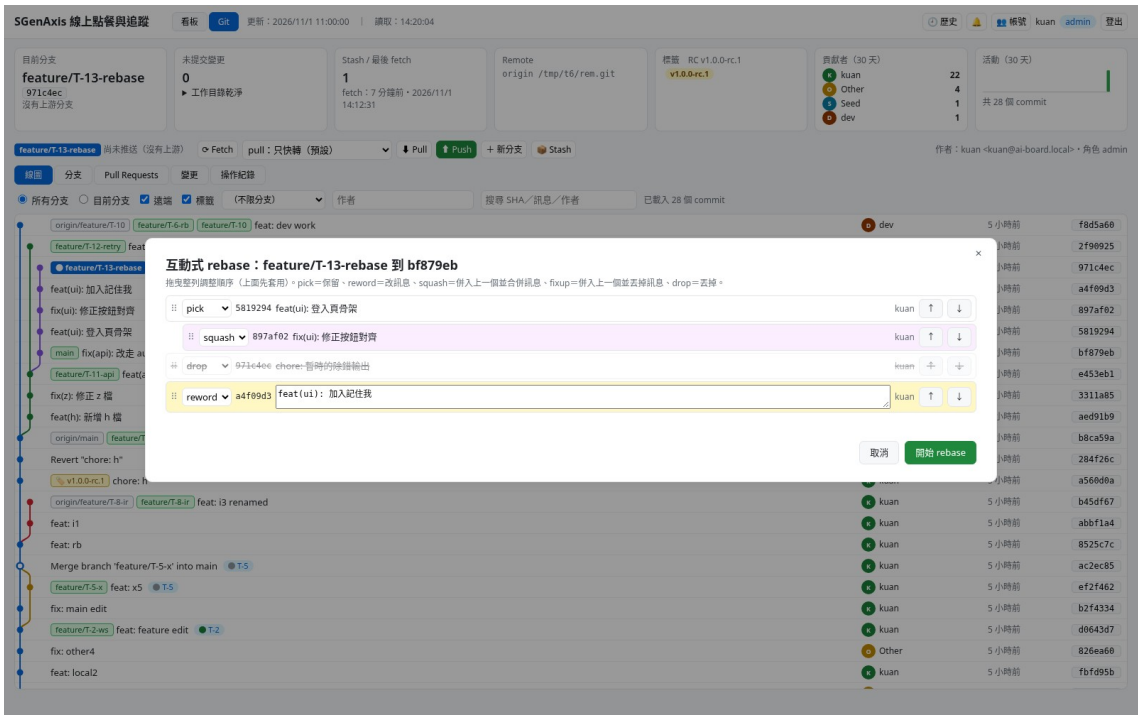


圖 4-11　互動式 rebase（拖曳排序、pick/reword/squash/fixup/drop）

User Stories

ID	User Story（作為…我想要…以便…）
US-14-1	作為開發代理，我想要在 feature 分支上 commit、push、rebase 未推送的 commit，以便完成工單。
US-14-2	作為產品負責人，我想要在看板上用三欄編輯器解衝突，以便不用切換工具。
US-14-3	作為產品負責人，我想要所有 Git 寫入都有稽核紀錄，以便追查。
US-14-4	作為產品負責人，我想要系統絕不 force push 或改寫已推送歷史，以便保護團隊的分支。

BDD 驗收情境

情境：開發代理只能在 feature 分支寫入	
假設 Given	角色 dev-agent，目前分支 main
當 When	嘗試 commit 或 push
那麼 Then	拒絕 403 E_ROLE_FEATURE_ONLY，並寫入稽核紀錄
情境：開發代理不能打標籤	
假設 Given	角色 dev-agent
當 When	執行 tag v1.3.0
那麼 Then	拒絕 403 E_ROLE_NO_TAG
情境：已推送 commit 不得 rebase	
假設 Given	互動式 rebase 範圍含已推送 commit
當 When	開始 rebase
那麼 Then	整個 rebase 不開始，回 E_PUSHED_REWRITE 並說明原因
情境：push 被拒不強推	
假設 Given	遠端有本機沒有的 commit
當 When	push
那麼 Then	回 E_NON_FF，提示先 pull；不 force push
情境：CSRF	
假設 Given	請求 Origin 不是 http://localhost:8765
當 When	POST /api/p/A/git/op
那麼 Then	回 403 E_CSRF
情境：衝突標記未清	
假設 Given	合併編輯器結果仍含 <<<<<<<
當 When	按「儲存並標記已解決」
那麼 Then	擋下（E_MARKERS），除非勾「仍要標記完成」
情境：非本機不可寫入	
假設 Given	**【現況】**請求不是來自本機（例如未來的手機遠端連線）
當 When	POST /git/op（任何 op）
那麼 Then	伺服器回 403 E_NOT_LOCAL；**【建議】**遠端唯讀 session 對其他寫入端點一律回 E_READ_ONLY

權限

- viewer：只能看（按鈕不顯示、伺服器也拒絕，E_ROLE_VIEWER）。
- dev-agent：只能在 feature/* 操作；不能推送或合併進主線、不能打標籤、不能刪除或改名別人的分支（看分支最新 commit 作者，E_ROLE_NOT_OWNER）；可 fetch 與 switch。

- verifier：dev-agent 能做的都可以，另可合併進非主線（受保護分支除外，E_ROLE_PROTECTED）。
- high-tier／admin：全部操作（仍受安全規則限制）。

桌機 vs 手機

平台	行為
桌機（完整操作）	全部 Git 寫入操作、合併編輯器、互動式 rebase、操作紀錄（圖 4-9～4-11）。
手機（監看）	**完全不提供**。手機只顯示操作結果（例如「T-4 分支已推送」事件）與唯讀的操作紀錄摘要 **【建議】**。

4.15 強制遵守機制（四層）

功能描述

只寫規則 AI 可能不照做，所以分四層把關，越後面越無法繞過：

- **第 1 層 規則**：技能與各工具規則檔。
- **第 2 層 AI Hooks（Cursor、Claude Code）**：`.ai-hooks/ai-hook.ps1` 共用一支腳本。擋危險指令（rm -rf /、git push --force、git reset --hard、format、curl|sh、iwr|iex...）、詢問 git push／clean／publish／DROP TABLE；禁止讀寫 .env、*.pem、*.key、id_rsa、secrets.json；改檔後格式化與規則檢查（unwrap、console.log、寫死金鑰）；結束前跑測試（cargo test／npm test，失敗要求續修，Cursor 最多續跑 3 次）。寫入 board.json 前用 validate-board.ps1 檢查；開發代理不能設 Done／藍燈／Released／approvedBy；不能改 .feature、hook、CI 設定；擋 --no-verify 與改 core.hooksPath；改了程式卻沒更新卡片會被要求補上；**檢查出錯一律阻擋（fail closed）**。日誌 .ai-hooks/logs/hooks.log（1 MB 輪替）。角色設在 config.json 的 role 或環境變數 AI_AGENT_ROLE（預設 dev-agent）。
- **第 3 層 Git Hooks（.githooks）**：pre-commit 檢查 board.json 規則與 updatedAt／history、新增 skip／only／#[ignore]、Cargo.toml／package.json／tauri.conf.json 版本同步（非測試 unwrap 只警告）；commit-msg 檢查 Conventional Commits。
- **第 3 層 CI Guard（.github/workflows/ai-dev-guard.yml）**：guard job：PR 必須對應一張 In Review＋TDD Green 的 Task、涵蓋情境 Green；改 .feature 要 spec-change 標籤；發版要改 CHANGELOG 並升版。tests job：跑 cargo test／npm test。
- **第 4 層 分支保護**（BRANCH-PROTECTION.md）：必要檢查沒過不能合併、需 1 位非開發代理審查、CODEOWNERS 保護 .feature／hook／CI、禁止 force push。
- 本機 `git commit --no-verify` 仍能跳過 Git Hooks（AI Hooks 會擋 AI 這樣做），但 CI 與分支保護一定檢查。

User Stories

ID	User Story（作為…我想要…以便…）
US-15-1	作為產品負責人，我想要 AI 無法自行把工單標完成，以便完成狀態可信。

ID	User Story（作為…我想要…以便…）
US-15-2	作為產品負責人，我想要 PR 沒對應到已驗證的 Task 就不能合併，以便每個變更都可追溯。

BDD 驗收情境

情境：開發代理改 .feature 被擋	
假設 Given	角色 dev-agent
當 When	代理嘗試修改 docs/bdd/S-1.feature
那麼 Then	AI Hooks 阻擋
情境：PR 未對應 Task	
假設 Given	PR 分支 feature/T-99-x，board.json 無 T-99 或 T-99 不在 In Review
當 When	CI Guard 執行
那麼 Then	guard 失敗，PR 不能合併（分支保護）
情境：驗證器自身錯誤	
假設 Given	validate-board.ps1 執行時拋出例外
當 When	AI 嘗試寫入 board.json
那麼 Then	一律阻擋（fail closed）

權限

- 角色值由使用者設定（config.json 或 AI_AGENT_ROLE）；AI 不得停用、修改或繞過 hook。

桌機 vs 手機

平台	行為
桌機（完整操作）	所有強制機制在桌機 AI 工具、本機 Git 與 GitHub 執行。
手機（監看）	不適用；手機可看到 CI Guard 結果（PR 檢查項目）。

5 資料模型

5.1 board.json（version 1）

根物件：`version`（常數 1）、`project`（≤200 字）、`updatedAt`（date-time）、`items`（≤2000 項）。範本：board.example.json；結構：board.schema.json（JSON Schema 2020-12）。

欄位	型別／限制	說明
id	^[A-Za-z0-9._-]{1,40}$，唯一	卡片 ID（E-1、F-1、S-1、T-1、D-1…）
level	Epic | Feature | Story | Task | Deployment	
title	≤300 字	
parentId	string | null	上層卡片
status	一般：Backlog | Todo | In Progress | In Review | Done Deployment：Planned | Deploying | Verifying | Released | Rolled Back	依 level 限制
light／lightReason	Gray | Green | Yellow | Red | Blue／≤200 字	黃紅必填原因；Done／Released ⇔ Blue；Rolled Back ⇒ Red
estimate	{storyPoints} | {hours}（≥0）	Story 不可有 hours；Task 不可有 storyPoints
owner、role、sprint	string	負責人、負責角色（開發／驗證）、Sprint
tdd、ci	None | Red | Green；None | Pending | Green | Red	
prUrl	^https?://	PR 連結（亦用於通知對應）
acceptance	string[]	驗收標準
bdd	[{id, title≤500, tests[≤50], status}]，≤50	Story 的 BDD 情境
scenarioIds	string[]，≤50	Task 涵蓋的情境 ID
notes／remark	[{at, by, text≤2000}] ≤100／≤2000 字	純文字顯示
createdAt、updatedAt、startedAt、completedAt	ISO 8601，必須含 Z 或 ±hh:mm	completedAt ≥ startedAt
history	[{at, from, to, by, note≤1000}] ≤200	移動卡片或改燈號時追加
Deployment 專用	version（SemVer，可含 -alpha/-beta/-rc.N）、releaseType、environment、strategy、rolloutPercent 0–100、featureFlags[≤100]、includes[≤100]、gitRef、commitSha（7–40 hex）、changelogUrl、buildUrl、healthChecks、rollbackPlan、rollbackRef、deployedAt、approvedBy	Deployment 必填 version、releaseType、environment；Prod 必填 rollbackPlan、approvedBy（不得為 dev-agent）；Hotfix 須為 Patch

跨項規則（validate-board.ps1）：ID 不重複、parentId 存在、上層燈號匯總一致性、同版本同環境不可重複、releaseType 與前一版升版相符、Story 完成需 BDD 全 Green 等。

5.2 設定檔（.ai-board，不進 Git）

檔案	內容	備註
users.json	本機帳號（PBKDF2 雜湊＋鹽）與 GitHub 帳號、角色、專案授權	從不對外提供
projects.json	專案清單：id、name、board.json 路徑、git repo、GitHub	最多 50 個

檔案	內容	備註
	repo、remoteSync	
pending.json	待核准的 GitHub 使用者	
github-oauth.json	{clientId}（8–64 英數）	不含 secret
git-audit.log	Git 寫入操作稽核	畫面顯示最近 200 筆
tmp\	commit 訊息等暫存檔	
.gitignore	*	整個資料夾不進 git

5.3 其他設定

檔案	重點欄位
.ai-hooks/config.json	enabled、blockDangerousShell、protectSecrets、formatOnEdit、guardOnEdit、stopTests、stopClippy、testCommands、testTimeoutSec(300)、toolTimeoutSec(60)、logMaxKB(1024)、role
.cursor/hooks.json	beforeShellExecution、beforeReadFile、afterFileEdit、stop
.claude/settings.json	PreToolUse、PostToolUse、Stop
.ai-dev-skills-manifest.txt	安裝檔案清單（移除依據）
serve-board.ps1 參數	-Port 8765、-Json、-Html、-ConfigDir、-MaxClients 20、-NoOpen、-NoAuth、-AutoCommit on|off、-DebounceMs 2000、-InitGit、-GitHubInterval 60、-SyncInterval 60、-GitHubApi、-GitHubWeb

6 API 規格

Base URL：`http://localhost:8765/`。回應為 JSON（錯誤格式為錯誤碼，例如 `{"error":"E_FORBIDDEN"}`，不含內部訊息）。所有 POST 須通過 CSRF 檢查（Origin 必須是 http://localhost:8765 或 http://127.0.0.1:8765）；一般 POST 本文上限 4 KB（E_TOO_LARGE），`/git/op` 上限 2 MB（GIT_OP_MAX）。只接受 GET／POST（其他回 405 E_METHOD）。「手機」欄為【建議】的手機唯讀 session 是否允許。

6.1 公開與驗證

方法	路徑	驗證	說明	手機
GET	/、/board.html	無	看板頁面	✓
GET	/api/status	無	{setup, noAuth, github}：是否需首次設定、是否關閉登入、是否已設 Client ID	✓
POST	/api/login	無	本機帳號登入（-NoAuth 時回 local/admin）	待決
POST	/api/logout	Cookie	登出並清除 Cookie	✓
POST	/api/setup	本機、未設定	建立第一位本機管理員	✗
POST	/api/setup/github	本機／admin	設定 GitHub Client ID	✗
POST	/api/gh/start	無	開始 Device Flow，回傳使用者代碼與驗證網址	✓
POST	/api/gh/poll	無	輪詢 Device Flow 結果，成功時建立 session（或進入待核准）	✓
GET	/api/me	登入	使用者、角色、可見專案（git、autoCommit、github、remoteSync、syncError）、待核准數（admin）	✓
GET	/events	登入	SSE：board／history／git／pr／sync；上限 20 連線	✓（前景）
GET／POST	/api/admin/users	admin	列出／核准／拒絕／改角色／改授權／移除帳號	✗
POST	/api/projects	admin	新增專案（必要時由範例建立 board.json 並 commit）	✗
GET	/board.json	登入	舊網址相容：轉到第一個可見專案	✓

6.2 專案 API（/api/p/{id}/...）

`{id}` 必須符合 `^[a-z0-9][a-z0-9-]{0,39}$`（否則 400 E_BAD_PROJECT）；專案不存在或無權限一律 403 E_FORBIDDEN。沒有 git 的專案，Git 類端點回 503 E_GIT_DISABLED。

方法	子路徑	參數	說明	手機
GET	/board.json	If-None-Match	看板資料；ETag／304	✓
GET	/history	limit(1–100,預設 30)、skip、id	board.json 的 commit 清單（可依卡片 ID 過濾）	✓
GET	/history/{sha}		某一版 board.json	✓
GET	/git/overview		概覽（分支、上游、變更數、標籤、貢獻者、活動）	摘要
GET	/git/graph	分支、篩選、分頁	線圖資料（每次 200 筆）	✗
GET	/git/log	ref 等	commit 清單	✗
GET	/git/info/{sha}		commit 詳細（JSON）	✗
GET	/git/commit/{sha}		commit 全文與 patch（text，上限 1 MB）	✗
GET	/git/filediff/{sha}	path、old	單檔 diff	✗

方法	子路徑	參數	說明	手機
GET	/git/branches		分支清單與狀態	✓
GET	/git/prs		PR、CI、審查（快取 60 秒）	✓
GET	/git/status		工作目錄變更與進行中狀態	✗
GET	/git/workdiff	path、staged=1	工作目錄 diff	✗
GET	/git/conflict	path	衝突檔三方內容	✗
GET	/git/rebase-plan	onto	互動式 rebase 計畫	✗
GET	/git/audit		操作紀錄（最近 200 筆）	摘要
POST	/git/op	{op, ...}	Git 寫入操作（見 4.14 表；**【現況】**只接受本機請求 E_NOT_LOCAL；角色檢查、單一佇列、逾時、稽核）	✗（現況已擋）

6.3 錯誤碼（節錄）

類別	錯誤碼
驗證／授權	E_UNAUTHENTICATED、E_FORBIDDEN、E_CSRF、E_AUTH、E_LOCKED、E_PENDING、E_DENIED、E_EXPIRED、E_ROLE、E_ROLE_VIEWER、E_ROLE_FEATURE_ONLY、E_ROLE_NO_TAG、E_ROLE_PROTECTED、E_ROLE_NOT_OWNER、E_LAST_ADMIN、E_SELF、E_NOT_LOCAL
輸入驗證	E_BAD_REQUEST、E_BAD_PROJECT、E_BAD_ID、E_BAD_PATH、E_BAD_REF、E_BAD_BRANCH、E_BAD_TAG、E_BAD_MESSAGE、E_BAD_MODE、E_BAD_OP、E_BAD_HUNK、E_BAD_STASH、E_BAD_TODO、E_BAD_NAME、E_BAD_ROLE、E_BAD_CLIENT_ID、E_WEAK_PASSWORD、E_TOO_LARGE、E_COMMIT_FORMAT、E_CONFIRM
Git 狀態	E_GIT_DISABLED、E_IN_PROGRESS、E_NO_STATE、E_DIRTY、E_DETACHED、E_CONFLICTS_LEFT、E_MARKERS、E_UNMERGED、E_HAS_MERGES、E_NOT_ANCESTOR、E_NO_BASE、E_NO_UPSTREAM、E_NO_REMOTE、E_MIRROR、E_NON_FF、E_NOT_FF_PULL、E_PULL_CONFLICT、E_PUSH_REJECTED、E_PUSHED_REWRITE、E_PROTECTED、E_BRANCH_EXISTS、E_TAG_EXISTS、E_TIMEOUT、E_BUSY
其他	E_ALREADY_SETUP、E_NO_CLIENT_ID、E_FLOW、E_GITHUB、E_GITHUB_UNREACHABLE、E_GITHUB_USER、E_NO_GITHUB_REPO、E_PROJECT、E_PROJECT_LOAD、E_NAME_TAKEN、E_SYNC、E_NOT_FOUND、E_METHOD、E_INTERNAL
【建議】新增	E_READ_ONLY（手機／遠端唯讀 session 嘗試寫入）、E_REMOTE_DISABLED（未啟用遠端存取時的非本機連線）

7 權限矩陣

✓＝允許；✗＝不允許；F＝僅 feature/* 分支；N＝非主線／非受保護分支；◯＝經 AI Hooks／CI 把關（看板本身不提供編輯）。手機欄適用所有角色（**【建議】**伺服器強制唯讀）。

動作	viewer	dev-agent	verifier	high-tier	admin	手機
檢視看板、Inspector、歷史	✓	✓	✓	✓	✓	✓唯讀
接收 PR／CI 通知	✓	✓	✓	✓	✓	✓
Git 瀏覽（概覽、線圖、PR）	✓	✓	✓	✓	✓	PR／CI、分支摘要
fetch、switch	✗	✓	✓	✓	✓	✗
commit、stash、push、rebase、reset（未推送）	✗	F	N	✓	✓	✗
merge 進非主線	✗	F	N	✓	✓	✗
推送／合併進主線、受保護分支	✗	✗	✗	✓	✓	✗
tag、tagPush	✗	✗	✗	✓	✓	✗
刪除／改名他人分支	✗	✗	N	✓	✓	✗
force push、改寫已推送 commit、--no-verify	✗	✗	✗	✗	✗	✗
修改 board.json（代理）	—	◯	◯	◯	◯	✗
設 Done／藍燈／Released／approvedBy	—	✗	◯ 待確認	✓	✓（使用者）	✗
修改 .feature／hook／CI	—	✗	✗	需核准＋spec-change	需核准	✗
帳號核准、角色與授權	✗	✗	✗	✗	✓	✗
新增專案、設定 Client ID	✗	✗	✗	✗	✓（本機）	✗
檢視操作紀錄	✓	✓	✓	✓	✓	摘要

註：verifier 能否設定 Done 依技能規則為「只有高階模型或使用者」，矩陣標「待確認」以與 AI Hooks 實作再核對。dev-agent 刪除／改名分支時依分支最新 commit 作者判斷是否為自己的分支。

8 安全性

8.1 現況控制

面向	控制
網路曝露	HttpListener 只綁 `http://localhost:<Port>/`；預設不對外。
驗證	GitHub OAuth Device Flow（無 client secret）；本機帳號 PBKDF2-SHA256 60 萬次＋16 byte 鹽、密碼 ≥ 12 字；連錯 5 次鎖 15 分鐘。
Session	32 byte 隨機 Cookie、HttpOnly、SameSite=Strict，伺服器只存 SHA-256；閒置 8 小時失效；上限 100。
CSRF	所有 POST 檢查 Origin 為本機。
授權（BOLA／BFLA）	每個 API 檢查專案授權與角色；不存在與無權限一律 403；專案只用 ID，從不接受用戶端路徑。
輸入驗證	專案 ID、卡片 ID、SHA、分支、路徑、標籤皆以正規式驗證；git 參數以 -- 分隔；POST 本文 4 KB 上限。
機密	GitHub token 只在伺服器記憶體；GITHUB_TOKEN 不送到瀏覽器、不寫入日誌；remote 網址去除帳密；.ai-board 整個 gitignore。
XSS	備註、歷程等所有文字當純文字顯示。
Git 安全	不 force push、不改寫已推送 commit、不 --no-verify、受保護分支不可刪、單一操作佇列、逾時、稽核紀錄。
錯誤外洩	只回錯誤碼；內部例外回 500 E_INTERNAL。
供應鏈	零相依；安裝檔未簽章（已知風險）。

與上線就緒標準的差異（已記錄理由）

production-readiness 建議密碼用 Argon2／BCrypt；看板因「零相依、PowerShell 內建」改用 PBKDF2-SHA256 60 萬次。屬有理由的偏離，審查時視為已核准例外。

8.2 手機／遠端存取的安全要求【建議】

- 預設維持只綁 localhost；遠端存取必須由使用者**明確選擇開啟（opt-in）**，畫面持續顯示「遠端存取已開啟」警示。
- 遠端連線一律 **HTTPS**；Cookie 加上 Secure；不得在明文 HTTP 上傳送 session。
- **【現況】** `/api/setup` 與 `/git/op` 已只接受本機請求（E_NOT_LOCAL），首位 GitHub 管理者也只能在本機產生；遠端 session 由伺服器標記為**唯讀**：所有 POST（login／logout／gh 流程除外）、/git/op、/api/admin/*、/api/projects、/api/setup* 一律拒絕（E_READ_ONLY），不只靠前端隱藏按鈕。
- CSRF 允許來源改為設定檔白名單（只加入遠端存取的固定網址），不接受任意 Origin。
- 首次設定、帳號管理、Client ID 設定只限本機（loopback）請求。
- 速率限制與鎖定沿用；遠端 session 閒置逾時建議縮短（例如 2 小時，**【待決】**）。
- 稽核：記錄遠端登入（時間、帳號、來源 IP）。
- 不得把看板直接以連接埠轉發曝露到公開網際網路。

9 非功能需求

類別	需求	來源
效能	board.json 變更到頁面更新 ≤ 1 秒（500 ms 檢查週期＋SSE）；自動 commit 防抖 2 秒；PR 輪詢 60 秒；線圖每頁 200 筆。	現況
容量上限	items ≤ 2000、history ≤ 200／卡、notes ≤ 100／卡、bdd ≤ 50、專案 ≤ 50、SSE ≤ 20、session ≤ 100、diff 1 MB、hooks.log 1 MB 輪替、🔔 100 筆、稽核畫面 200 筆。	現況
可靠性	board.json 格式錯誤時保留上一版；Git 操作單一佇列＋逾時（2／5 分鐘）；遠端同步失敗停下不強推；AI Hooks fail closed。	現況
相容性	Windows 10／11、Windows PowerShell 5.1 與 PowerShell 7；Git for Windows；瀏覽器 Chrome／Edge（file:// 監看需 Chrome／Edge）；macOS／Linux 需把 hook 設定的 powershell 改為 pwsh。**【未驗證】**PS 5.1 實機。	現況／未驗證
零相依	伺服器只用 PowerShell／.NET 內建；頁面單一 HTML；降級可用 python／npx 靜態伺服。	現況
無障礙	卡片可 Tab 選取、Enter 開 Inspector、Esc 關閉；燈號不可只靠顏色（附文字／數字）；**【建議】**手機點擊目標 ≥ 44×44 px、文字對比 ≥ 4.5:1、支援系統字級放大。	現況／建議
國際化	介面為繁體中文（台灣）；時間 ISO 8601 含時區，顯示本地時間與相對時間；主控台 UTF-8（chcp 65001）。**【建議】**字串集中以便日後加英文。	現況／建議
可觀測性	主控台日誌（board 變更、commit 失敗、GitHub 輪詢失敗）；hooks.log；git-audit.log。**【建議】**結構化日誌與 trace ID。	現況／建議
手機效能**【建議】**	首次載入 ≤ 3 秒（4G）；背景不保持 SSE；總覽資料量 ≤ 100 KB（只送摘要）。	建議

10 手機監看版設計【建議】

本章全部為**【建議】**，依產品負責人「桌機完整操作、手機只監看」的要求設計。現況 board.html 有 viewport meta，但**沒有任何 @media 響應式規則**，且伺服器只綁 localhost，因此手機版需要前端與伺服器兩邊的新工作。

10.1 設計原則

- **唯讀**：手機上不出現任何編輯、Git 寫入、帳號或專案管理入口；伺服器端同步強制（E_READ_ONLY）。
- **先回答三個問題**：有沒有東西卡住（紅／黃）？最近發生什麼（通知）？PR／CI 狀態如何？
- **深連結**：所有通知與清單項目都能開唯讀卡片；卡片頁提供「在桌機開啟」的連結（複製網址）。
- 參考市面產品的監看模式（例如 GitHub Mobile 的通知收件匣、Jira Cloud 手機版的看板唯讀檢視），不另外發明互動。

10.2 斷點與版面

寬度	模式	版面
≥ 1024 px	桌機（完整操作）	現有五欄看板、Inspector 側欄、Git 分頁全部功能
768–1023 px	平板	**【待決】**建議視同監看模式（唯讀），但使用雙欄：左清單、右唯讀詳細
< 768 px	手機（監看）	單欄；底部分頁列：總覽／通知／卡片／PR·CI

判斷依據建議以「伺服器 session 類型（本機／遠端唯讀）」為主、視窗寬度為輔：遠端 session 不論寬度一律唯讀；本機的窄視窗只改版面、不改權限。

圖 10-1　手機監看版線框（建議）



圖 10-1　手機監看版線框（總覽／通知／唯讀卡片）

10.3 手機功能清單

分頁	內容	資料來源
總覽	專案切換；五燈號計數（可點展開清單）；需注意清單（紅 → 黃，附原因與更新時間）；各欄數量條；最新佈署（每個環境最新版本、狀態、放量 %）	/api/me、/api/p/{id}/board.json
通知	🔔 最近 100 筆（PR 開啟／合併／關閉、CI 變更、燈號轉紅／黃【建議新增事件】、佈署狀態），未讀標記；點擊開唯讀卡片或 PR 頁	SSE pr 事件、本機儲存
卡片	搜尋 ID／標題；唯讀詳細：狀態、燈號與原因、估算、TDD／CI、PR、BDD x/y、最近歷程、備註（純文字）	board.json、/history?id=
PR·CI	開啟中 PR、CI 結果與失敗檢查項目、審查結果、對應卡片；分支摘要（燈號、領先／落後、閒置）	/git/prs、/git/branches
不提供	Git 線圖、diff、所有 Git 寫入、合併編輯器、互動式 rebase、帳號管理、新增專案、首次設定	—

10.4 通知

類型	說明	條件
頁面內（現況可沿用）	SSE 即時更新 🔔 未讀數與清單；頁面回到前景時以 ETag 補抓	頁面開啟中
Web Push【建議】	Service Worker＋VAPID；伺服器在 pr／燈號轉紅事件時推送；通知內容只含 ID、標題與狀態，不含程式碼或機密	必須 HTTPS；iOS 需加入主畫面（PWA）
通知偏好【建議】	依專案、依事件類型（CI 失敗、PR 合併、燈號轉紅、佈署）開關；勿擾時段	存在使用者設定

10.5 手機登入

- 主要：GitHub Device Flow（手機上開 github.com/login/device 輸入代碼）；登入後 session 標記為遠端唯讀。
- 替代【建議】：桌機產生一次性配對 QR Code（短效、單次使用），手機掃描後取得唯讀 session，免輸入。
- 本機密碼帳號是否允許從手機登入【待決】（建議預設不允許，降低暴力破解面）。
- 遠端 session 閒置逾時建議 2 小時【待決】；可在桌機「👥 帳號」看到並撤銷手機 session【建議】。

10.6 遠端存取方案（LAN／Tunnel）

現況伺服器只綁 localhost。以下方案皆為【建議】，最終選擇列為【待決】（第 14 章）。

方案	做法	優點	風險／成本
A. 選用 LAN 綁定＋HTTPS＋Token	新參數（例如 -Remote lan）綁定內網位址；需 HTTPS 憑證（自簽或 mkcert，以 netsh http add sslcert 綁定）與 URL ACL；Windows 防火牆只開「私人」網路；配對 Token／QR；IP 白名單（選用）	不經第三方；延遲低	需管理員權限設定 urlacl／憑證；手機需信任自簽憑證；只能在同一網路
B. Tunnel（建議優先評估）	Tailscale（tailnet 內存取，可用 Tailscale Serve 提供 HTTPS）或 Cloudflare Tunnel＋Access（身分驗證在前）；伺服器仍只綁 localhost	不開放對外埠；有現成 HTTPS 與身分層；外出也能看	依賴第三方服務與帳號；需安裝代理程式
C. 只轉發通知	不開放看板，只把通知經 Web Push 或第三方推播送到手機	曝露面最小	看不到看板與卡片詳細；需推播服務

建議：外出監看採 B（Tailscale），同一 Wi-Fi 才用 A；兩者都必須搭配 8.2 的唯讀 session 與 HTTPS。

10.7 手機監看 BDD

情境：手機只能唯讀	
假設 Given	遠端存取已開啟，使用者以手機登入（唯讀 session）
當 When	送出 POST /api/p/A/git/op（push）或 POST /api/admin/users
那麼 Then	/git/op 回 403 E_NOT_LOCAL（現況）；/api/admin/users 回 403 E_READ_ONLY（建議）；稽核紀錄寫入被拒操作
情境：手機不顯示操作入口	
假設 Given	手機寬度 390 px 開啟看板
當 When	瀏覽任何分頁
那麼 Then	畫面上沒有任何編輯、Git 操作、帳號管理按鈕
情境：CI 失敗推播	
假設 Given	已訂閱 Web Push、T-4 對應 PR #42

當 When	#42 CI 由 Green 變 Red
那麼 Then	手機收到推播「PR #42 CI 失敗・T-4」，點擊開啟 T-4 唯讀卡片
情境：未開啟遠端存取	
假設 Given	伺服器未啟用遠端存取
當 When	手機嘗試連線
那麼 Then	無法連線（只綁 localhost）
情境：背景省電	
假設 Given	手機開著看板後切到背景
當 When	5 分鐘後回到前景
那麼 Then	SSE 重新連線並以 ETag 補抓，畫面為最新

11 測試策略

11.1 原則

- 依 ai-dev-pipeline：每個 Story 先寫 BDD（高階模型撰寫、使用者核准），每個情境對應測試；Task 以 Red → Green → Refactor 實作並保存失敗與通過輸出；開發代理不得修改 .feature、削弱斷言或 skip。
- 依 production-readiness：核心邏輯覆蓋率 ≥ 80%，領域層趨近 100%；每個 Adapter（檔案、git CLI、GitHub API）至少一組整合測試；主要流程 E2E 對應 BDD。
- 驗收：獨立驗證代理重跑情境並實際操作介面、截圖錄影；平台不符記 NOT RUN，不得偽造 PASS。

11.2 測試層級

層級	對象	工具（建議）
單元	validate-board 規則、燈號匯總、角色權限函式（Test-GitPerm）、輸入驗證正規式、時間戳規則	Pester（PowerShell）；JS 純函式用 Vitest 或瀏覽器內測試【建議】
Schema	board.schema.json 與 validate-board.ps1 一致性（同一組正例／反例檔案）	Pester＋JSON Schema 驗證器
整合	HttpListener API（登入、授權 403、CSRF、ETag 304、SSE 事件）、git 操作（暫存 repo）、GitHub API（以 -GitHubApi 指向假伺服器）	Pester＋臨時 git repo；假 GitHub 伺服器
E2E	看板五欄、篩選、Inspector、歷史、Git 操作、合併編輯器、互動式 rebase；手機 390×844 監看版	Playwright（含行動裝置模擬）【建議】
安全	BOLA（跨專案）、BFLA（角色越權）、CSRF、路徑穿越、參數注入、XSS（備註）、暴力破解鎖定、手機唯讀強制	Pester／Playwright 負向情境
安裝	install／uninstall 冪等、.bak 備份與還原、-Tools 子集、舊版遷移	Pester（暫存資料夾）；exe 以 Windows VM 實測

11.3 CI 關卡

關卡	內容	失敗時
AI Dev Guard / guard	PR 對應 In Review＋TDD Green 的 Task、涵蓋情境 Green；.feature 變更需 spec-change 標籤；發版需 CHANGELOG 與升版	不能合併
AI Dev Guard / tests	cargo test／npm test	不能合併
pre-commit／commit-msg	board.json 規則、skip／only、版本同步、Conventional Commits	本機 commit 被擋
【建議】看板自身 CI	Windows runner 跑 Pester（PS 5.1 與 PS 7 兩種）、Playwright（桌機＋手機尺寸）、PSScriptAnalyzer	不能合併

12 部署與安裝

12.1 建置

- `build_payload.py`：從 skills、hooks_src、enforce_src、board_src、templates_src 產生 payload\<元件>\。
- `gen_nsi.py`：產生 installer.nsi（元件勾選、預設路徑、捷徑、移除程式、舊版遷移）→ makensis 產出 dist/AI-Dev-Skills-Setup.exe。
- PowerShell 版：install.ps1、uninstall.ps1、payload 打包為 dist/AI-Dev-Skills-PowerShell.zip。
- 指南：dist/guide.html（安裝後位於 docs\ai-dev-skills\guide.html）。

12.2 安裝與首次使用流程

6. 執行 exe（或 install.ps1）→ 勾選 AI 工具 → 選資料夾（預設 D:\AI_INSTRUCTIONS）→ 完成後開指南。
7. 複製到專案：`robocopy D:\AI_INSTRUCTIONS D:\AIAD /E /XF uninstall.exe .ai-dev-skills-manifest.txt /XO`，或直接 `install.ps1 -ProjectPath D:\AIAD`。
8. 啟用 Git Hooks：`scripts\install-git-hooks.ps1`（設定 core.hooksPath=.githooks）。
9. 在 GitHub 設定分支保護（BRANCH-PROTECTION.md）：必要檢查 AI Dev Guard / guard、tests；1 位非開發代理審查；禁止 force push。
10. 雙擊「AI Dev 看板」→ 網頁完成首次設定（Client ID／本機管理員）→ GitHub 登入 → 新增專案。
11. 設定 AI 角色：.ai-hooks/config.json 的 role 或 AI_AGENT_ROLE。

12.3 升級與移除

- 升級：直接執行新版安裝；變動檔案備份後覆蓋，相同略過；舊版 .ai-board 自動遷移。
- 移除：uninstall.exe／uninstall.ps1；只刪清單內檔案、還原 .bak；.ai-board 保留。

12.4 手機監看上線（建議）

- 第 14 章決定遠端方案後，新增伺服器參數與唯讀 session，預設關閉。
- 指南新增「手機監看」章節：Tailscale 或 LAN 設定步驟、憑證、配對、通知訂閱。

13 已知限制與未驗證項目

#	項目	狀態	影響／後續
L-1	尚未在 Windows／Windows PowerShell 5.1 實機完整測試（伺服器、安裝檔、移除、捷徑、chcp 65001 顯示）	**【未驗證】**	需在 Windows 10／11 VM 跑完 11.2 安裝與 E2E 測試
L-2	尚未對真實 GitHub 測試 Device Flow、PR／CI 輪詢、ETag、速率限制、待核准流程	**【未驗證】**	建立測試 OAuth App 與測試 repo 實測
L-3	互動式 rebase 依賴 Windows Git 的 sh（GIT_SEQUENCE_EDITOR 等）在 Windows 上的行為	**【未驗證】**	於 Git for Windows 實測 pick／reword／squash／fixup／drop
L-4	Inspector 開啟時可能覆蓋頁首右側按鈕（🔔、帳號、歷史等）	已知問題	修正版面（Inspector 從頁首下方開始或頁首按鈕避讓）
L-5	board.html 無 @media 響應式規則；手機版面未實作	限制	依第 10 章實作
L-6	伺服器只綁 localhost，手機無法連線	設計限制	依第 10.6 方案決定
L-7	安裝檔未程式碼簽章，SmartScreen 警告	限制	評估簽章憑證
L-8	只安裝 Cursor 與 Claude Code 的 AI Hooks；其他工具只有規則與技能；Copilot CLI 讀 .claude/settings.json 但未測	限制	後續版本評估
L-9	本機 git commit --no-verify 可跳過 Git Hooks（CI 與分支保護仍檢查）	設計限制	依賴分支保護
L-10	GitHub token 只在記憶體，伺服器重啟需重新登入	設計取捨	安全優先
L-11	未驗證的 GitHub API 每小時 60 次上限	限制	建議登入或設定 GITHUB_TOKEN

14 待決事項

#	議題	選項	建議	決定者
Q-1	completedAt 由誰設定？	(a) 代理移到 Done 時自動填 (b) 只有高階模型或使用者 (c) 伺服器依 history 推算	(b)：與 Done／藍燈權限一致；AI Hooks 擋 dev-agent 寫 completedAt	Kuan Yu
Q-2	佈署版本唯一性範圍	(a) 每個環境唯一（現況：version@environment）(b) 整個看板唯一	(a)：同一版本須依序經 Dev → Staging → Prod，應允許跨環境重複	Kuan Yu
Q-3	手機遠端存取方式	A. LAN＋HTTPS＋Token　B. Tunnel（Tailscale／Cloudflare）C. 只推播通知	B 為主、A 為輔；皆需唯讀 session	Kuan Yu
Q-4	平板（768–1023 px）定位	監看或完整操作	監看（唯讀）	Kuan Yu
Q-5	本機密碼帳號可否從手機登入	允許／不允許	不允許（只用 GitHub 或 QR 配對）	Kuan Yu
Q-6	遠端 session 閒置逾時	8 小時（同桌機）／2 小時	2 小時	Kuan Yu
Q-7	Web Push 是否納入第一版	納入／延後（先只做頁面內通知）	延後：先完成唯讀監看與頁面內通知，再評估推播	Kuan Yu
Q-8	verifier 能否設 Done	依技能「只有高階模型或使用者」／允許 verifier	依技能：不允許，並與 AI Hooks 實作核對	Kuan Yu
Q-9	新增燈號轉紅／黃事件到通知	新增／不新增	新增（手機監看核心價值）	Kuan Yu

附錄 A 名詞解釋

名詞	說明
Epic／Feature／User Story／Task	工作層級：戰略目標／中型模組／使用者價值切片（SP）／技術細節（小時，一輪做完）
SP（Story Point）	故事點，只用於 Story
BDD	行為驅動開發；以 Given／When／Then（假設／當／那麼）描述情境（.feature）
TDD	測試驅動開發；Red（失敗）→ Green（通過）→ Refactor（重構）
READY_FOR_TDD	開發代理完成工單、等待獨立驗收；看板 In Review
RESLICE_REQUIRED／SPEC_CONFLICT／REPLAN_REQUIRED	工單需重切／規格矛盾／需重新規劃，由高階模型處理
健康燈號	Gray／Green／Yellow／Red／Blue，表示未開始／順利／風險／阻擋／完成
Inspector	看板右側卡片詳細面板
SSE	Server-Sent Events，伺服器單向推送事件
ETag	資源版本標記，用於判斷是否變更（304 Not Modified）
OAuth Device Flow	裝置授權流程：顯示代碼，使用者到 github.com/login/device 輸入授權
CSRF	跨站請求偽造；以 Origin 檢查防護
BOLA／BFLA	物件層級／功能層級授權失效（OWASP API Top 10）
PBKDF2	密碼雜湊演算法（本產品用 SHA-256、60 萬次）
Squash and Merge	把 PR 的多個 commit 壓成一個合併進 main
Revert	建立反向 commit 撤銷變更，不改寫歷史
Force push	強制覆蓋遠端歷史；本產品一律禁止
Cherry-pick	把某個 commit 套用到目前分支
Interactive rebase	互動式重整未推送的 commit（pick／reword／squash／fixup／drop）
Canary／Gradual Rollout	先給少量使用者（1%）／分階段放量（5% → 50% → 100%）
Feature Flag	以設定開關功能；需有負責人與移除日期
SemVer／RC	語意化版本 Major.Minor.Patch／發布候選版（-rc.N）
Hotfix	從正式版 tag 拉分支修復重大 Bug，版本為 Patch
Conventional Commits	commit 訊息格式，例如 feat(scope): 說明
Tunnel	經第三方服務建立的安全通道（例如 Tailscale、Cloudflare Tunnel），不需開放對外埠
Web Push／PWA	瀏覽器推播通知／可加入主畫面的網頁應用